Departamento de Informática

Bacharelado em Ciência da Computação

CI1212 Arquitetura de Computadores

Construindo o Processador MIPS Multiciclo

Controle

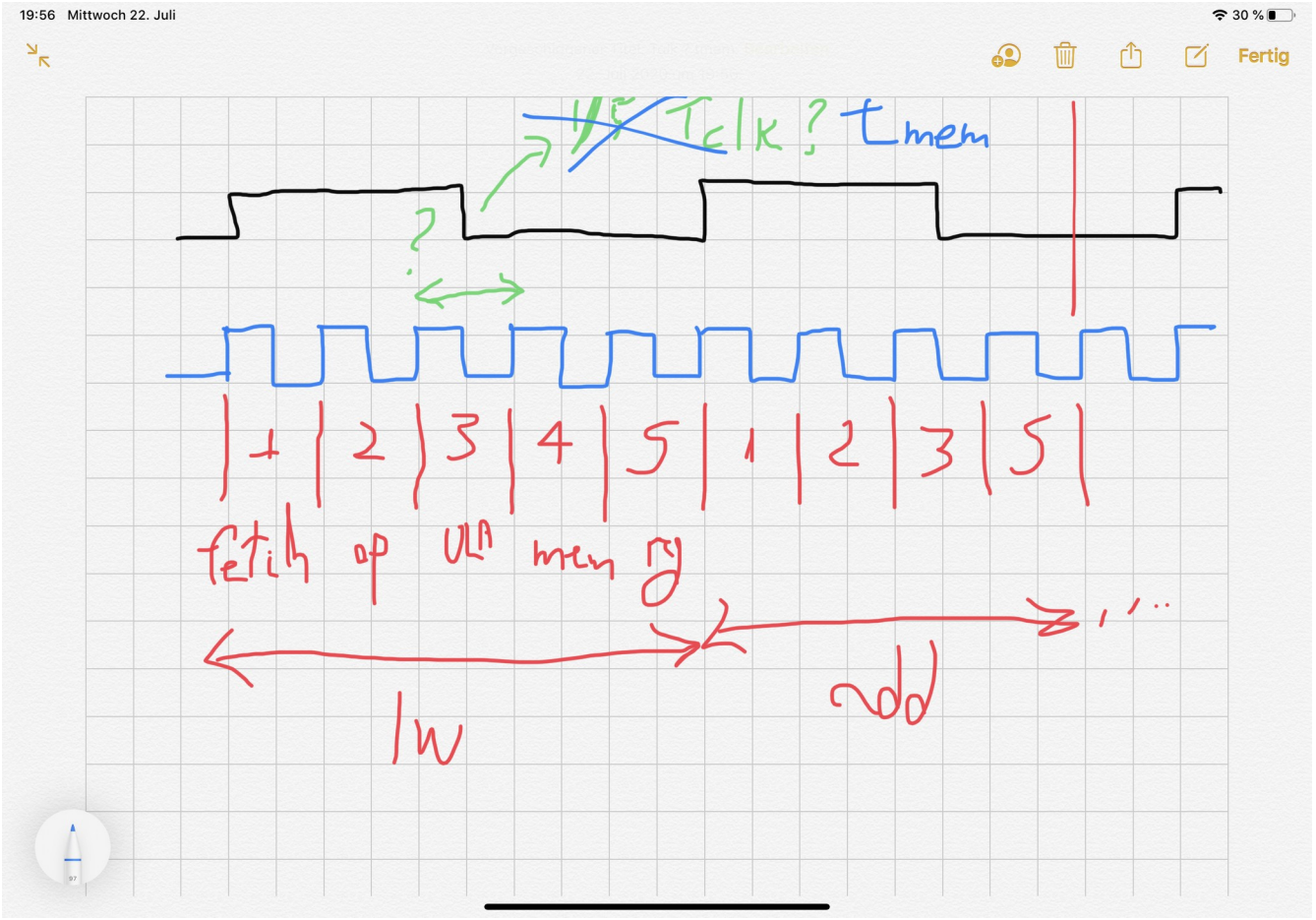
PE-2020
Prof. Eduardo Todt
Versão 202007221921

MIPS Multiciclo

Objetivos

Entender esquema de controle para o MIPS multiciclo

Lembre que dividimos a execução em partes, em até 5 ciclos de clock



Partes da execução

1 Instruction fetch

2 Instruction decode and register fetch

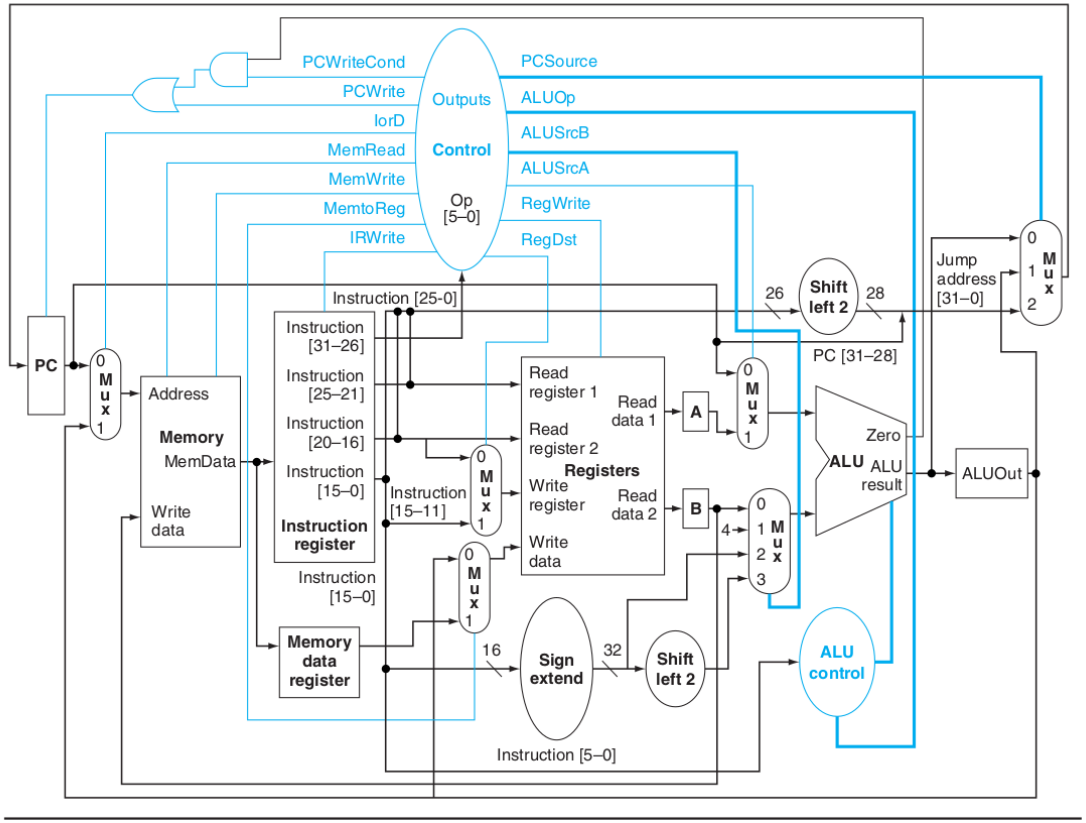
3 Execution, memory address computation, or branch completion or jump

4 Memory access (lw/sw), R-type instruction completion

5 Memory read completion step (lw)

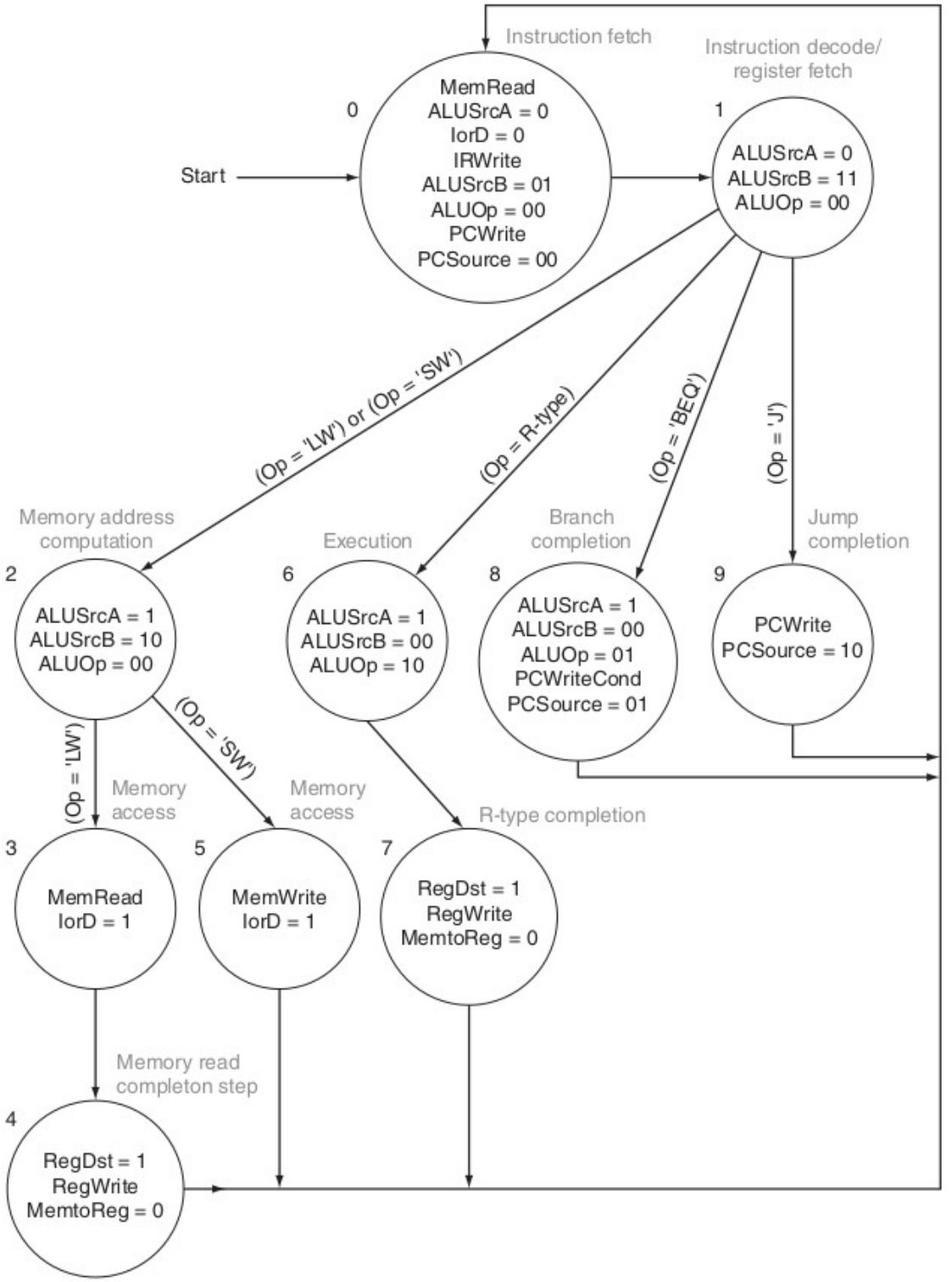
Controle

A cada um dos 3 a 5 ciclos devem ser ativados os sinais de controle necessários para realizar a operação desejada



Controle:

Máquina de Moore

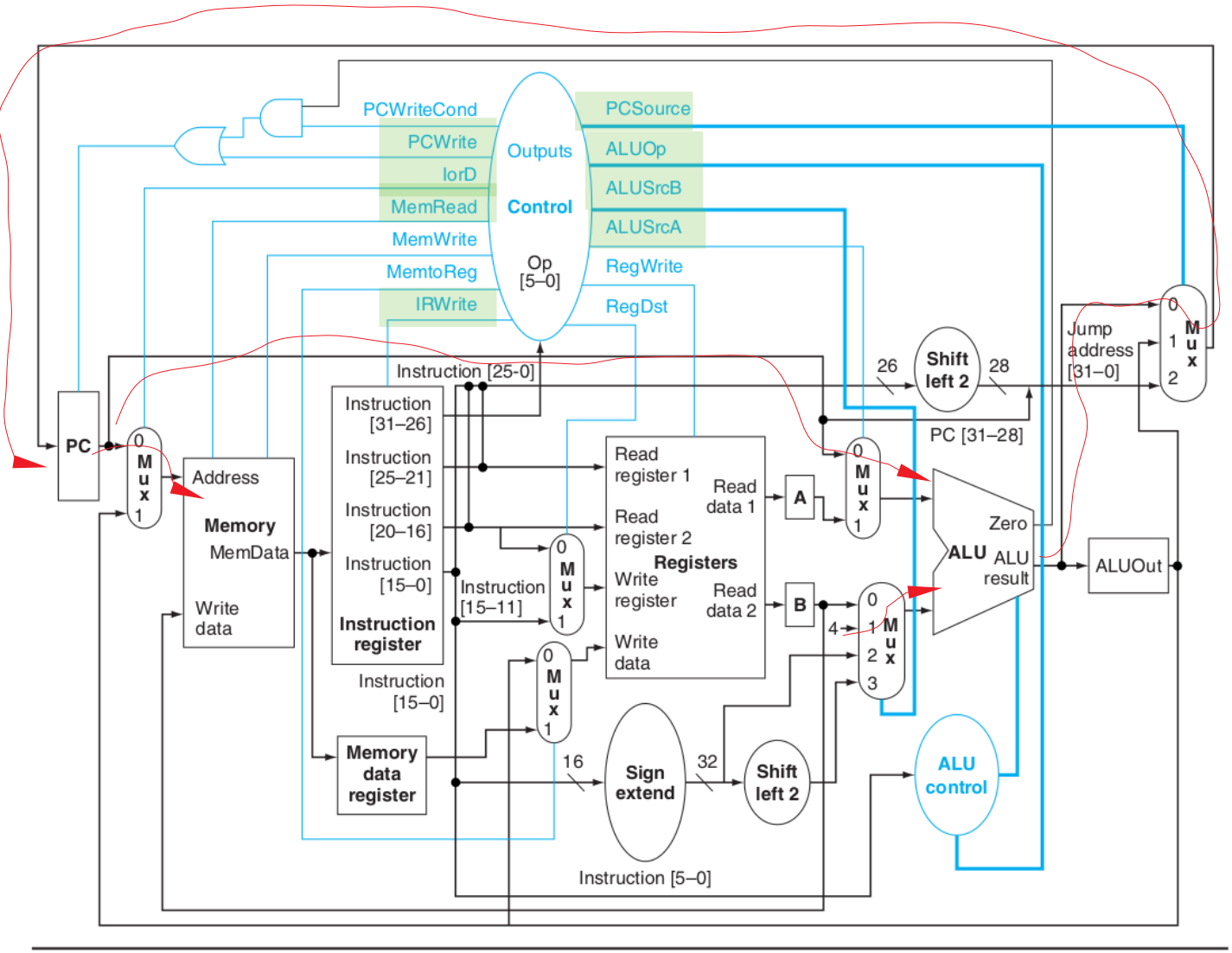


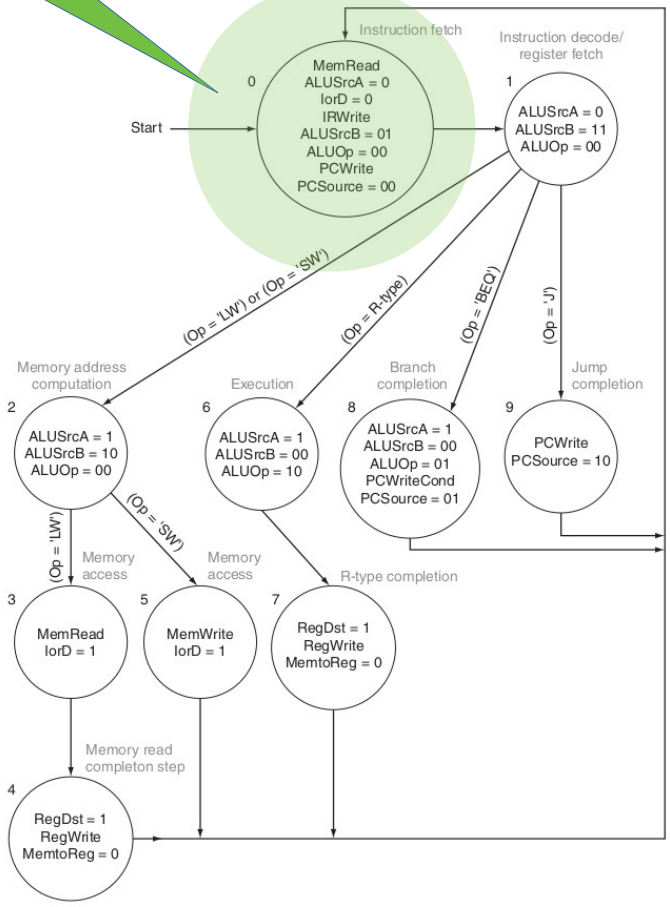
Vamos ver cada caso

add $1, $2, $3

add
1. Fetch

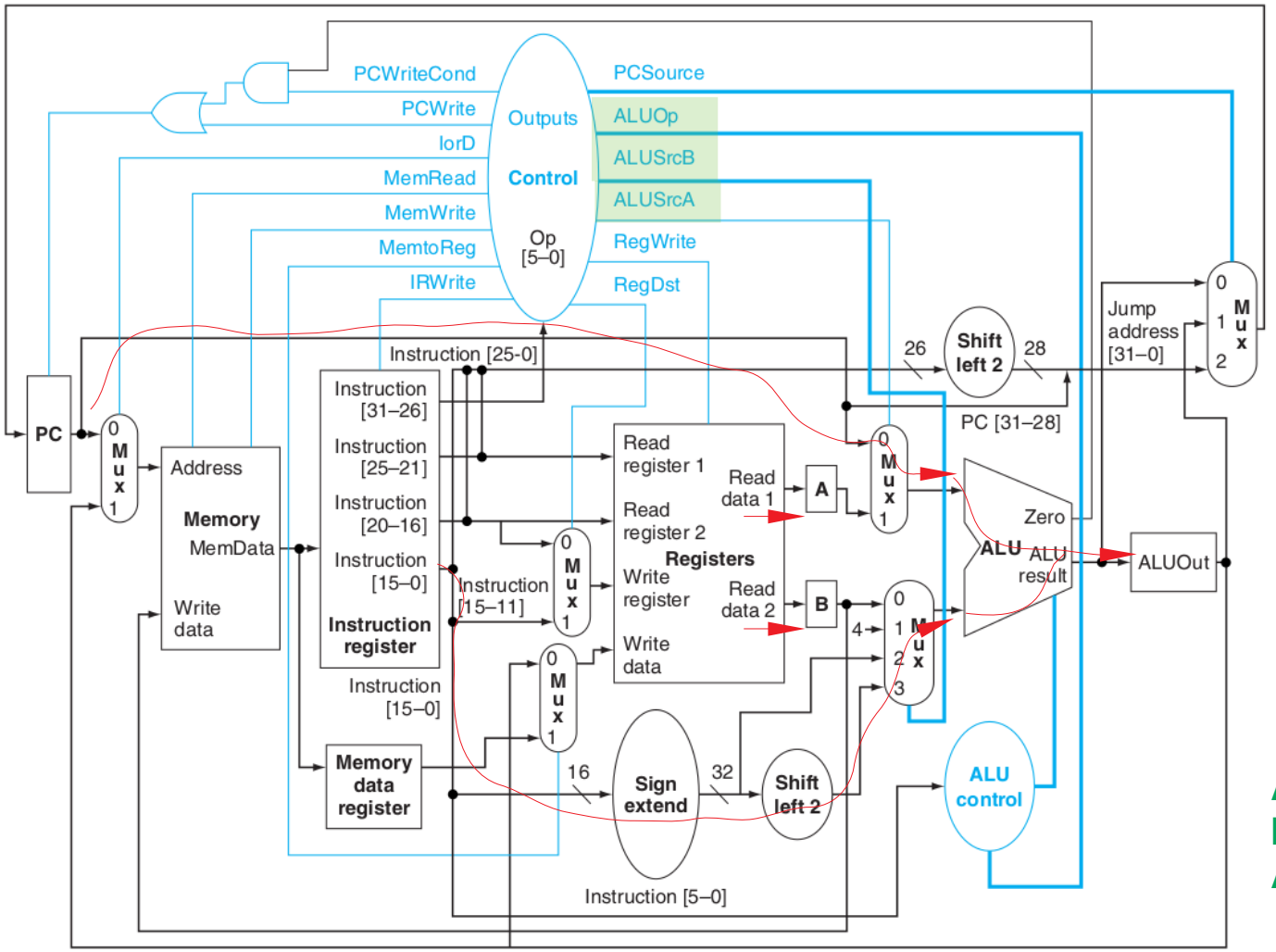
único ciclo em
que escreve no IR
Já o PC pode
ser escrito
de novo em
um desvio

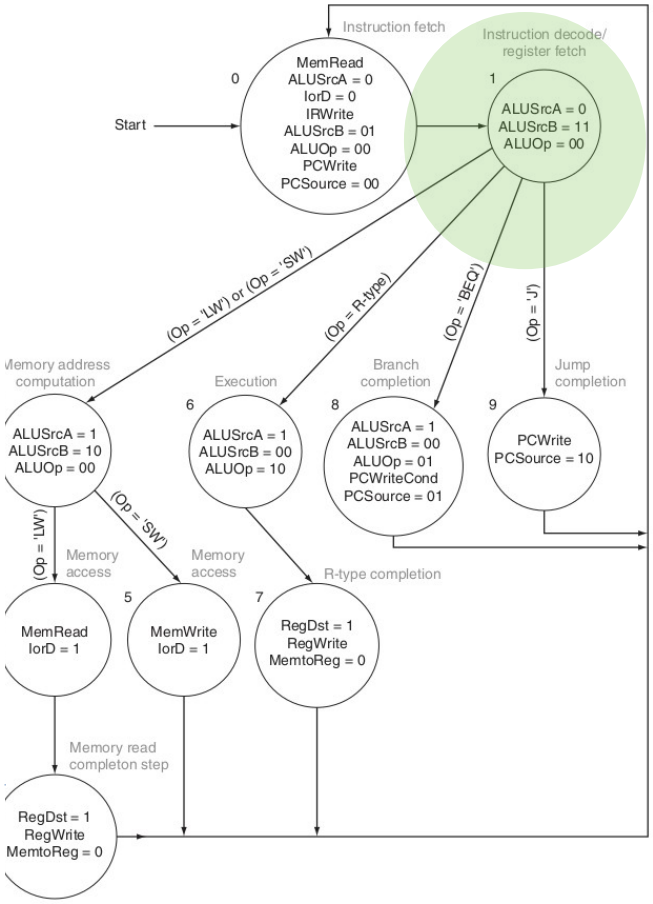




IR <= Memory[PC];
PC <= PC + 4;

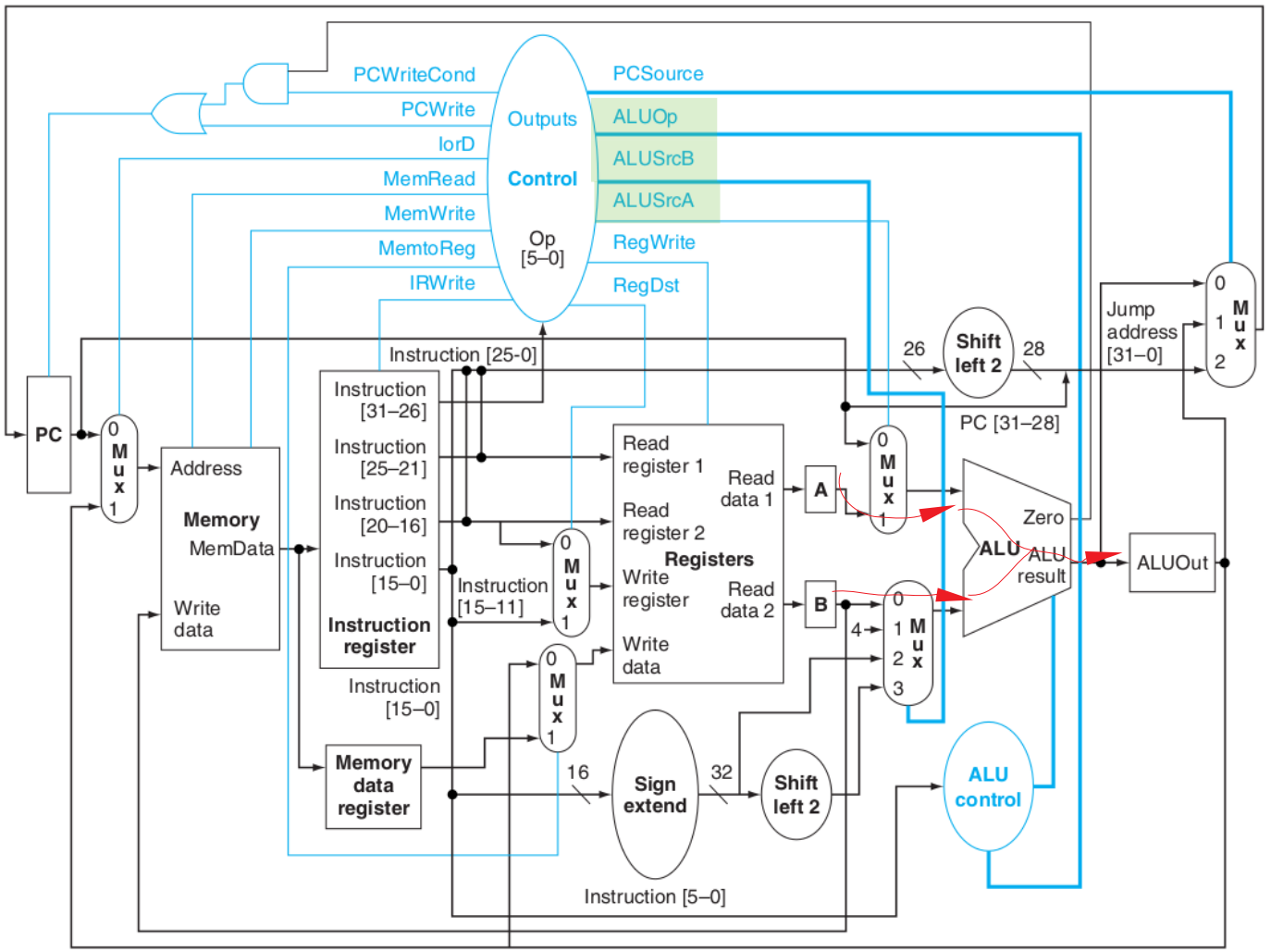
add
2. Decode and operand fetch

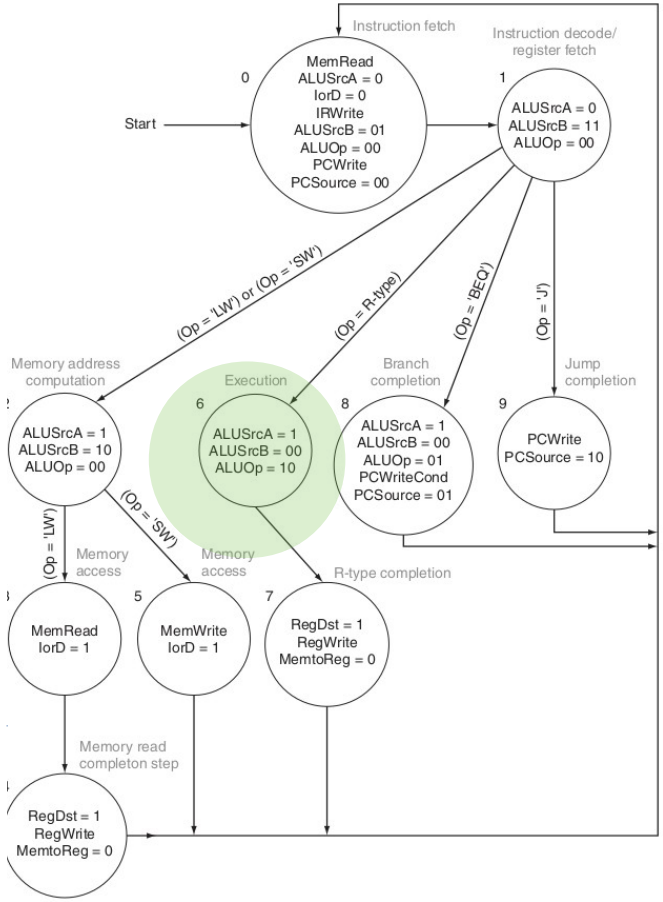





```
A <= Reg[IR[25:21]];
B <= Reg[IR[20:16]];
ALUOut <= PC +
      (sign-extend (IR[15-0]) << 2);
```

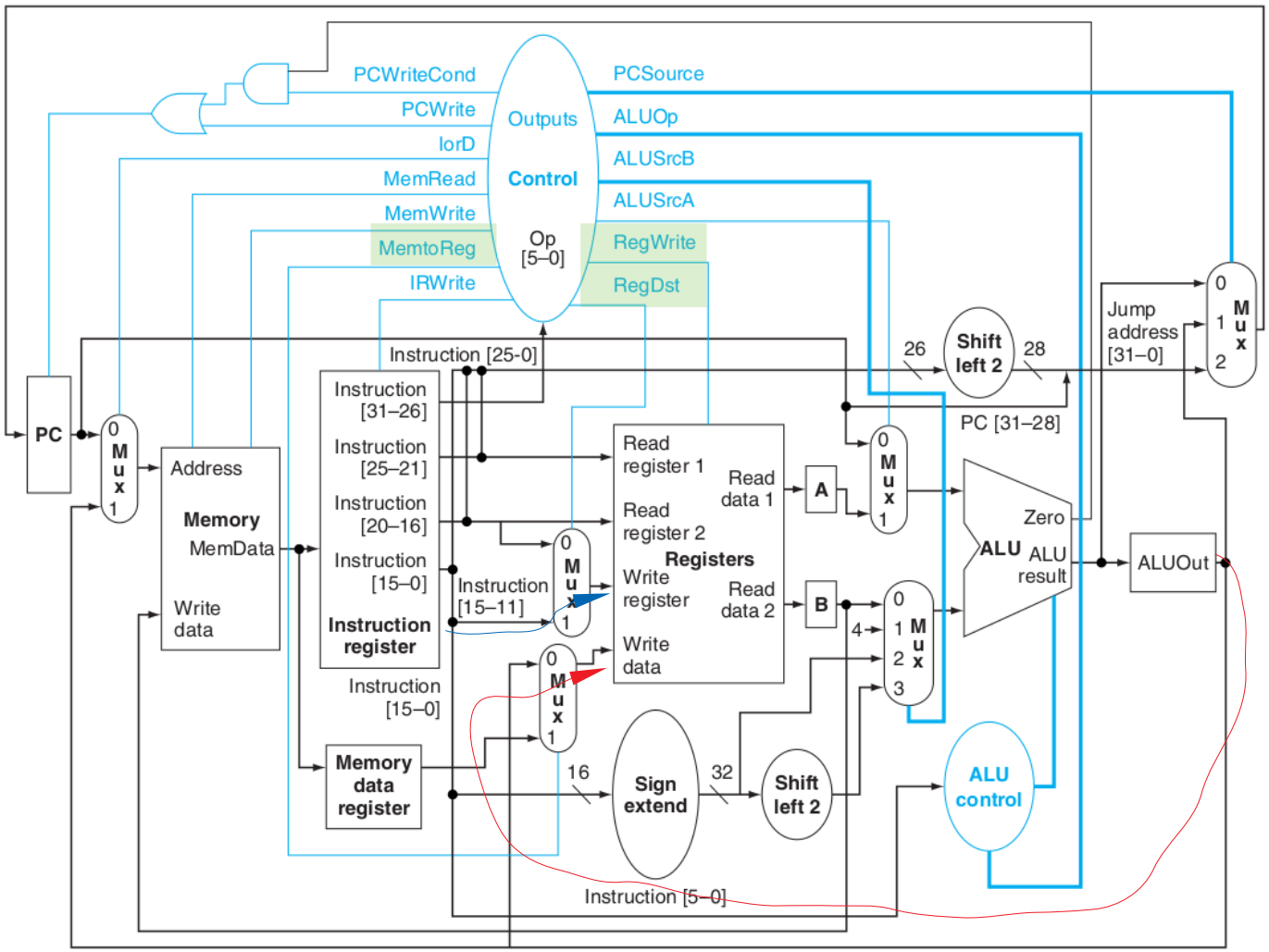
add
3. Execution

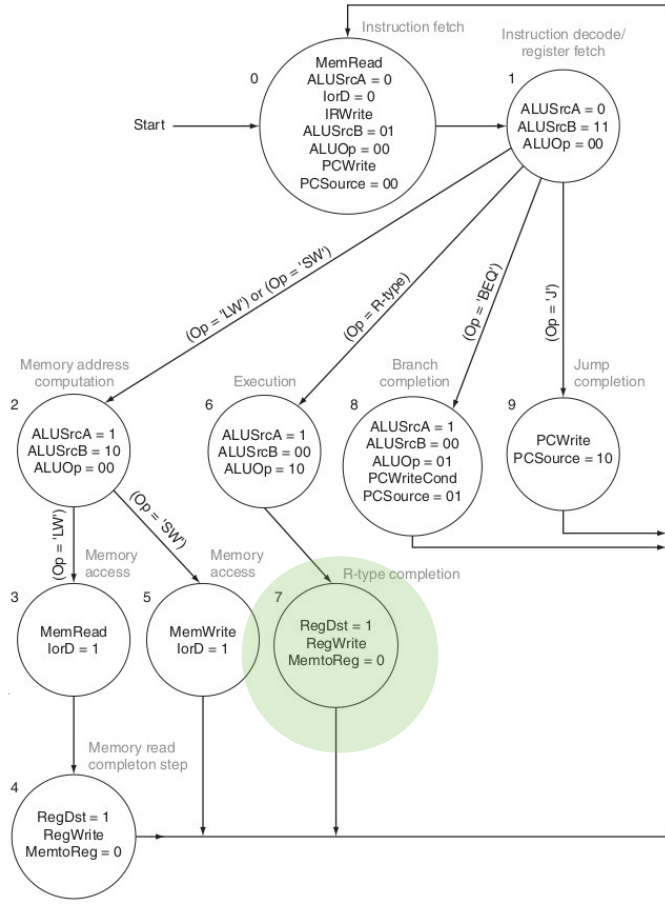




ALUOut <= A op B;

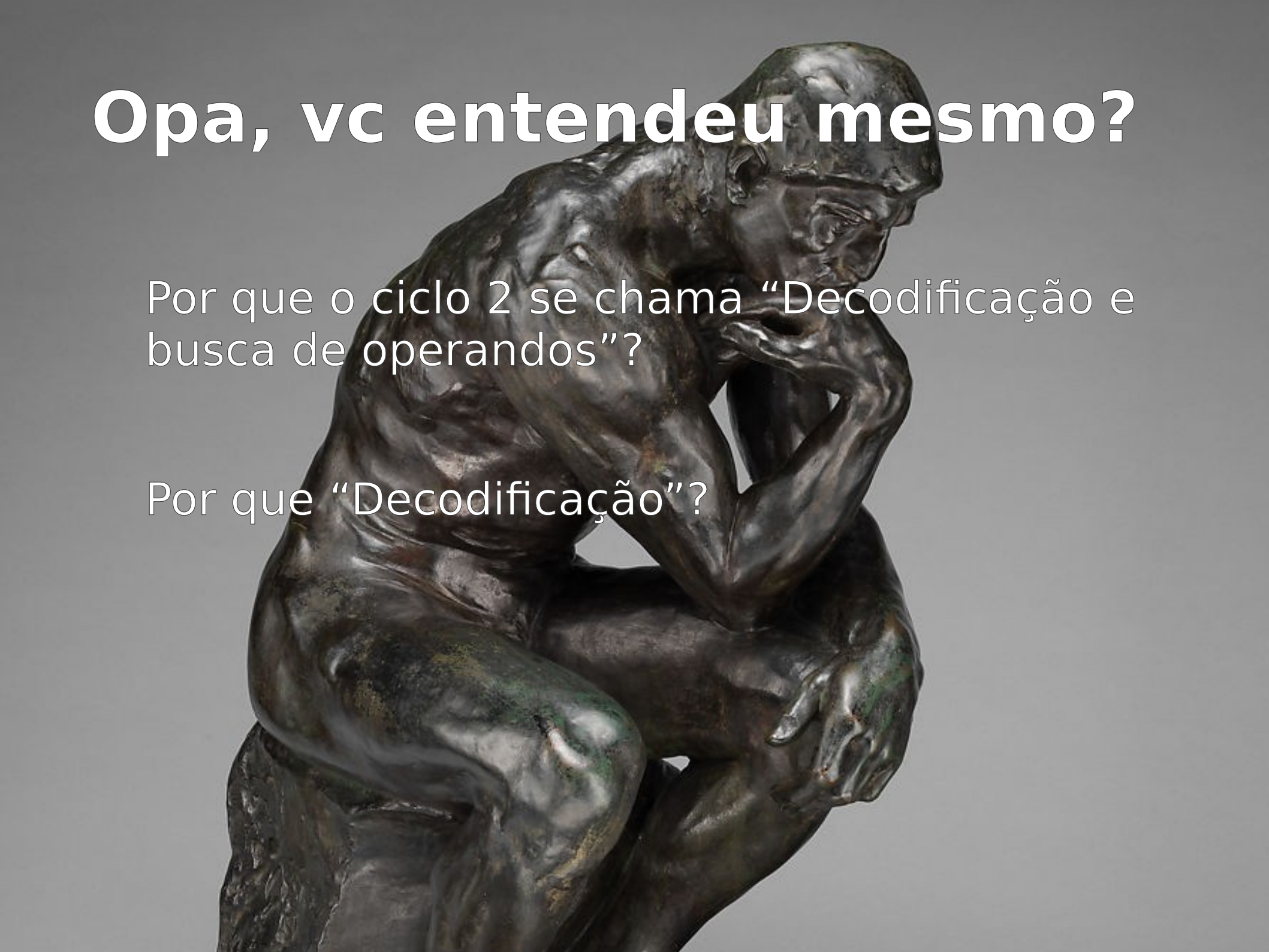
add
4. Instruction completion





Reg[IR[15:11]] <= ALUOut;

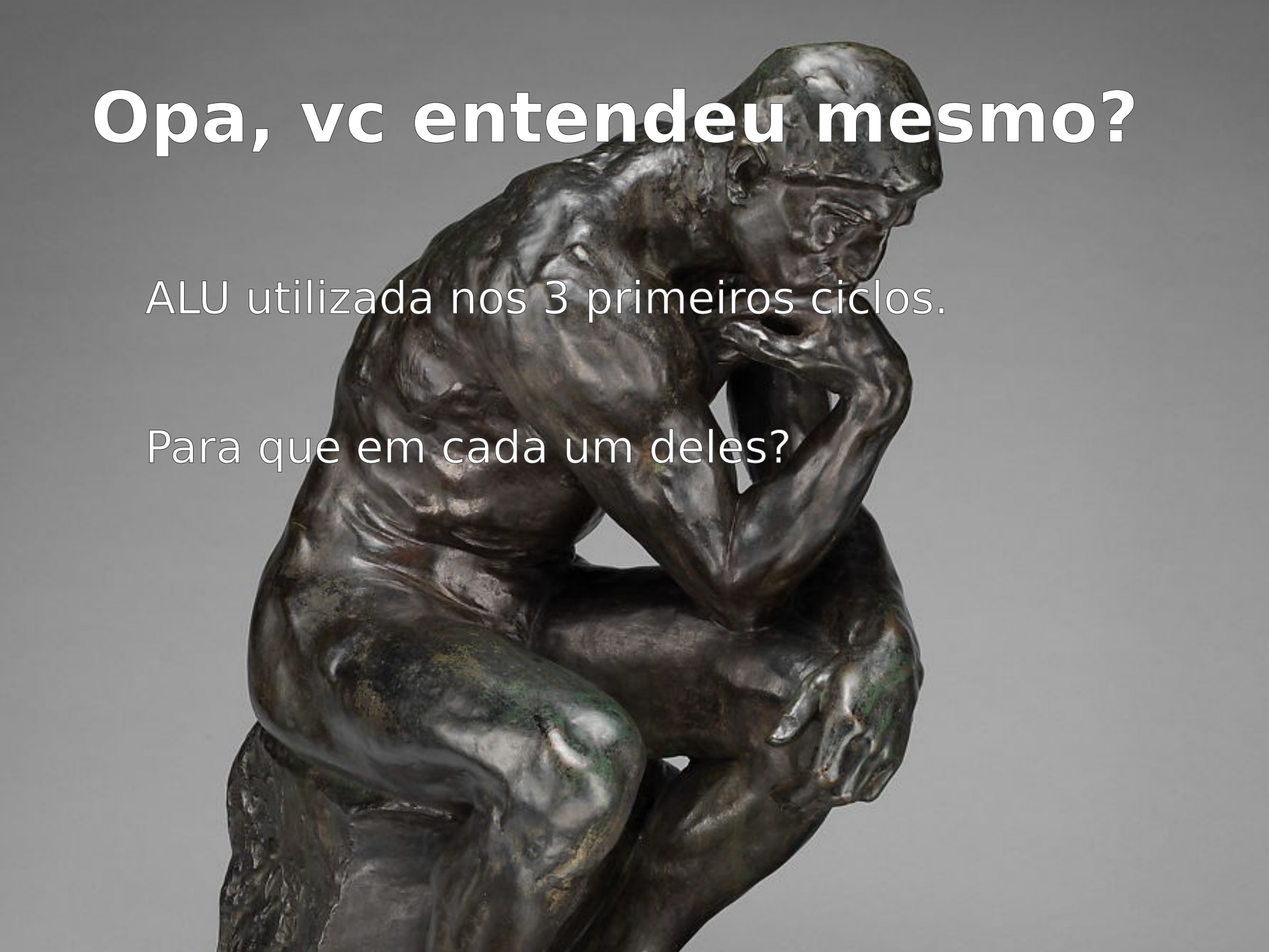
Opa, vc entendeu mesmo?



Por que o ciclo 2 se chama “Decodificação e busca de operandos”?

Por que “Decodificação”?

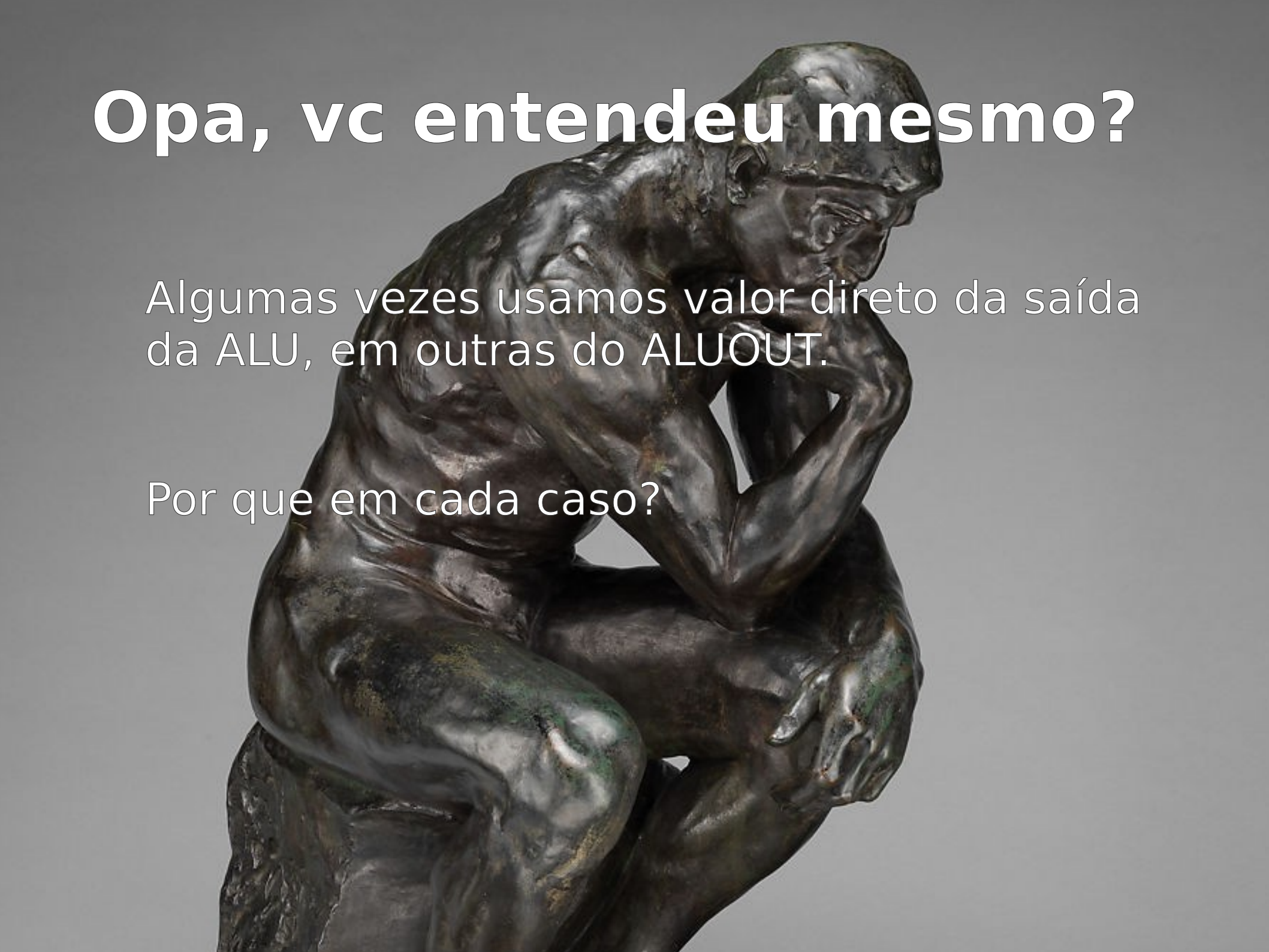
Opa, vc entendeu mesmo?



ALU utilizada nos 3 primeiros ciclos.

Para que em cada um deles?

Opa, vc entendeu mesmo?



Algumas vezes usamos valor direto da saída da ALU, em outras do ALUOUT.

Por que em cada caso?

Opa, vc entendeu mesmo?

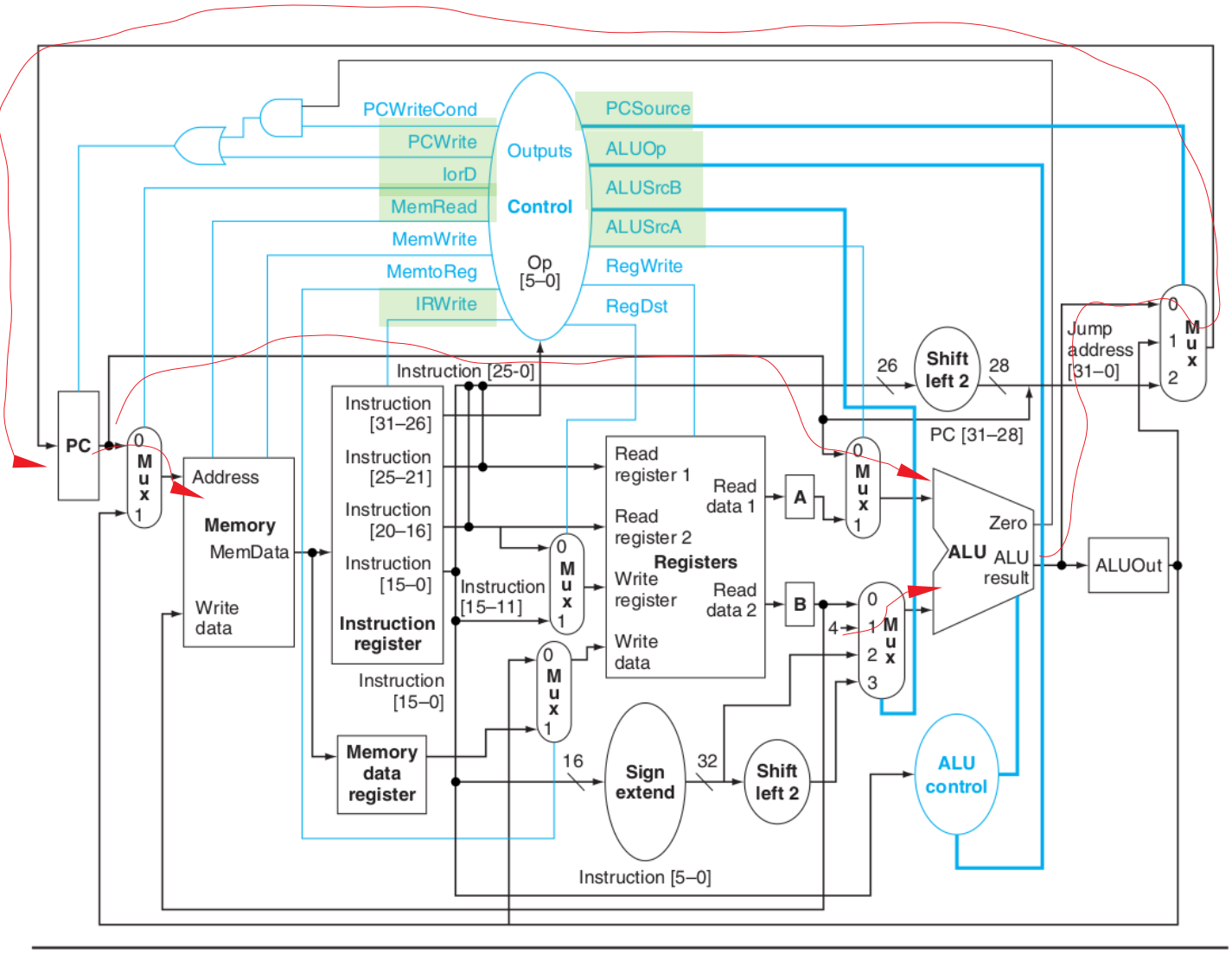
Conhece esta obra?

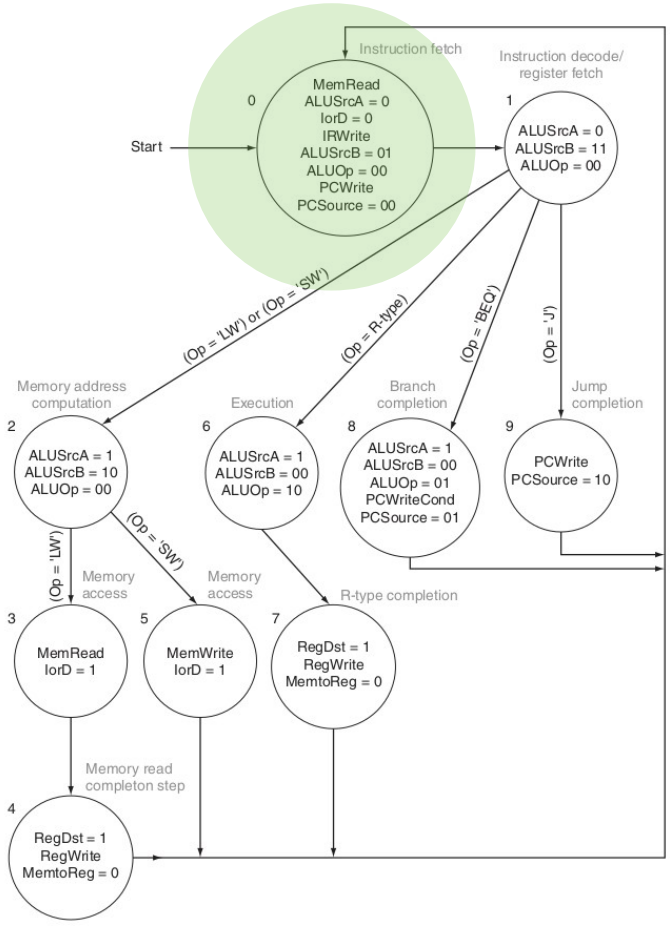
Qual autor?

https://www.metmuseum.org/art/collection/search/191811

beq $1, $2, 0x16

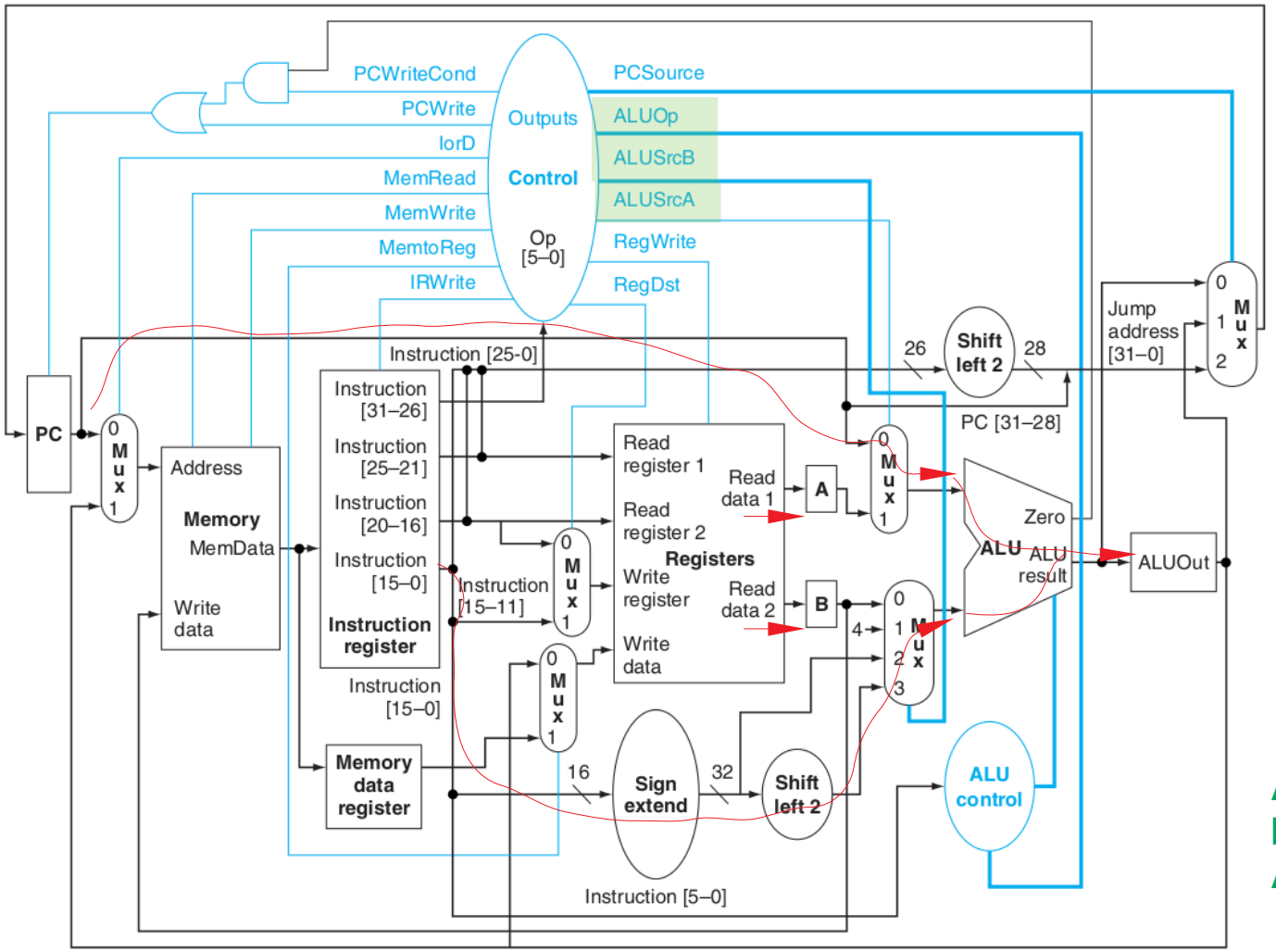
beq
1. Fetch

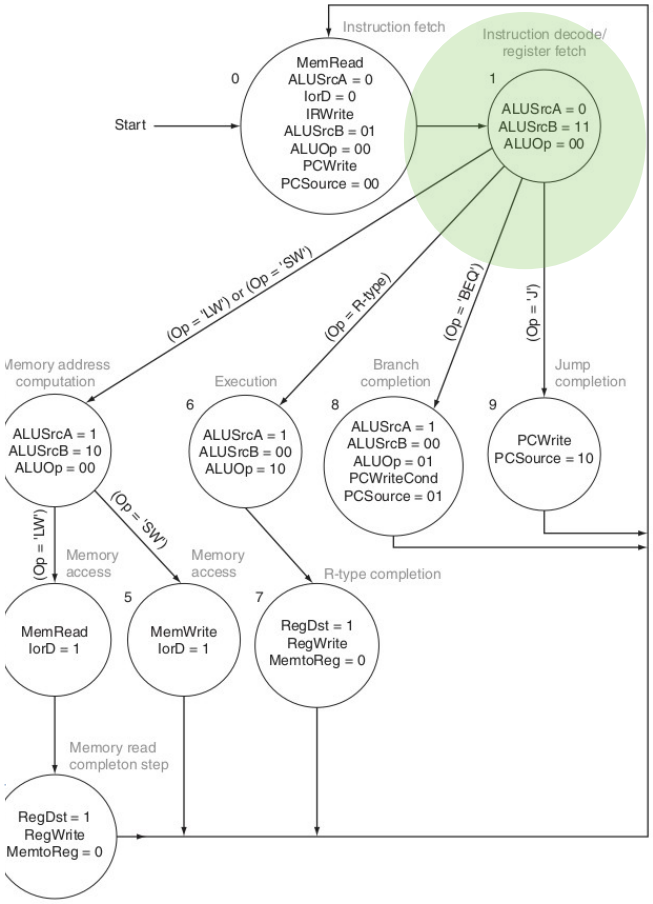




IR <= Memory[PC];
PC <= PC + 4;

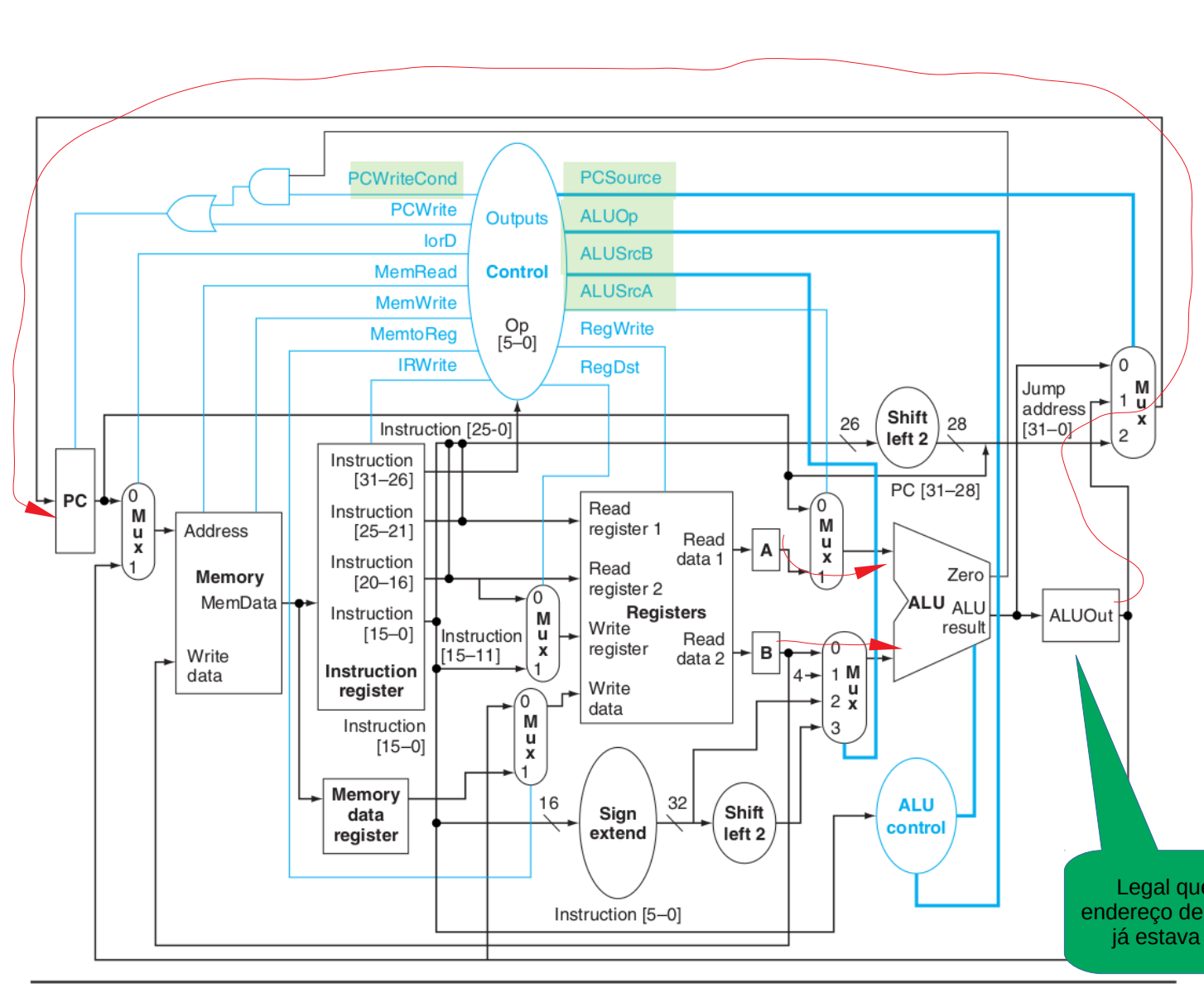
beq
2. Decode and operand fetch

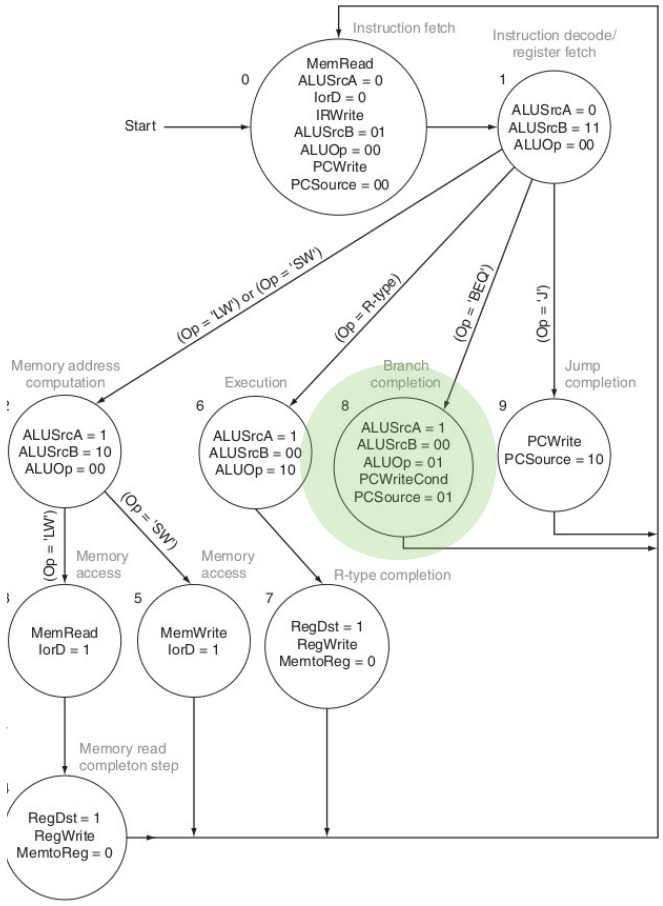





```
A <= Reg[IR[25:21]];
B <= Reg[IR[20:16]];
ALUOut <= PC +
      (sign-extend (IR[15-0]) << 2);
```

beq
3. Branch



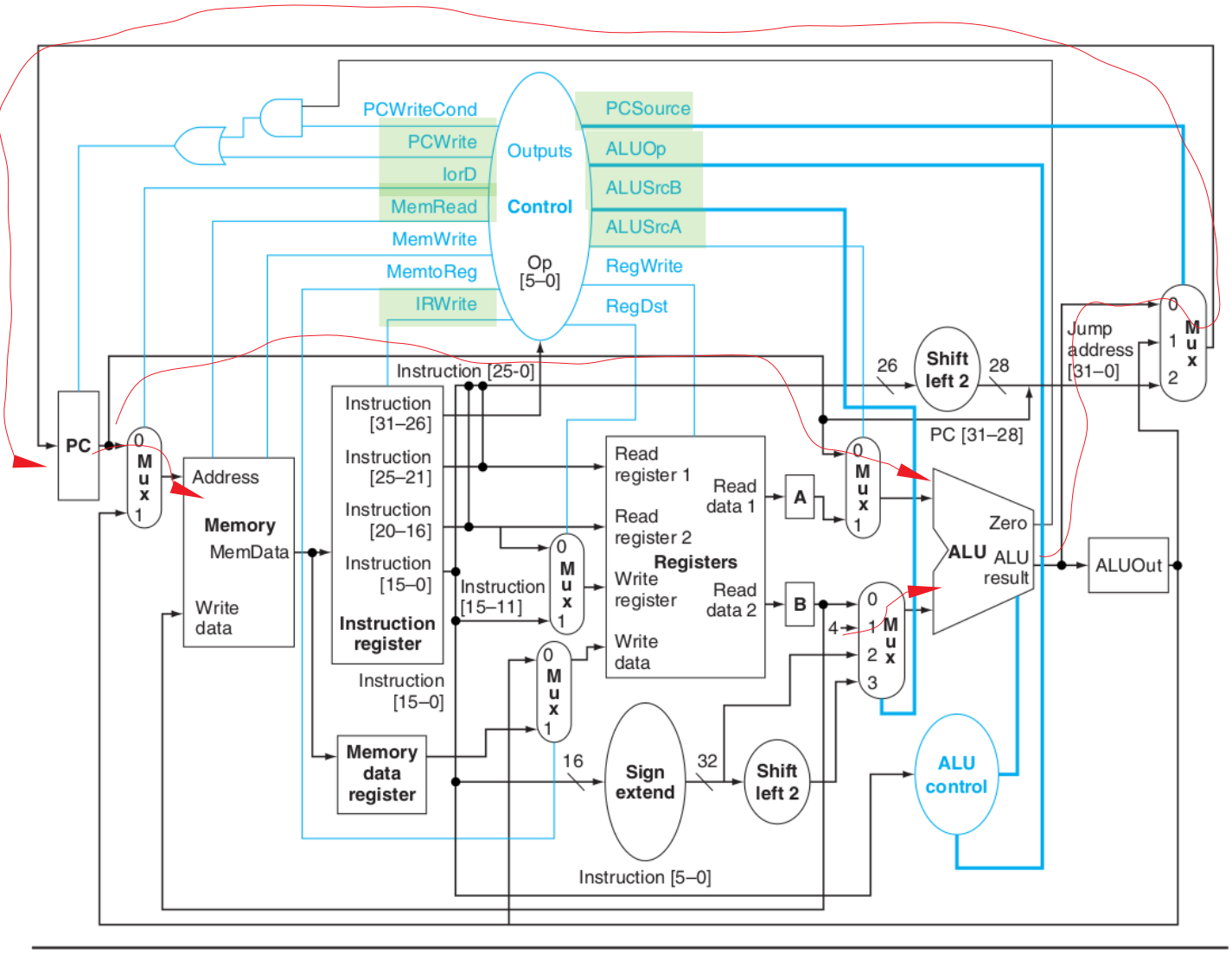


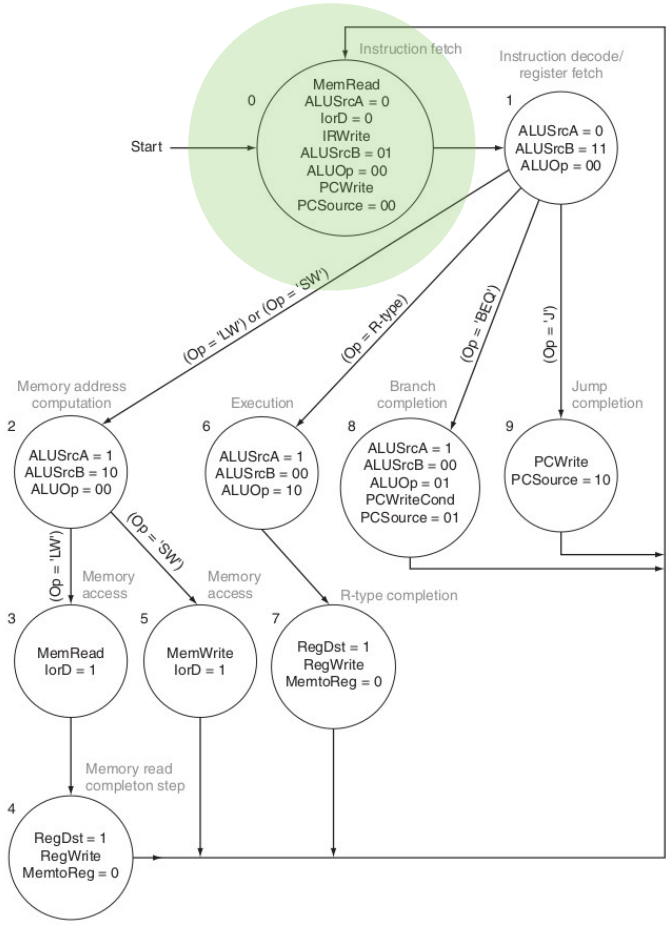
if (A == B) PC <= ALUOut;

Legal que o endereço de desvio já estava aqui

sw $1, 0x10($2)

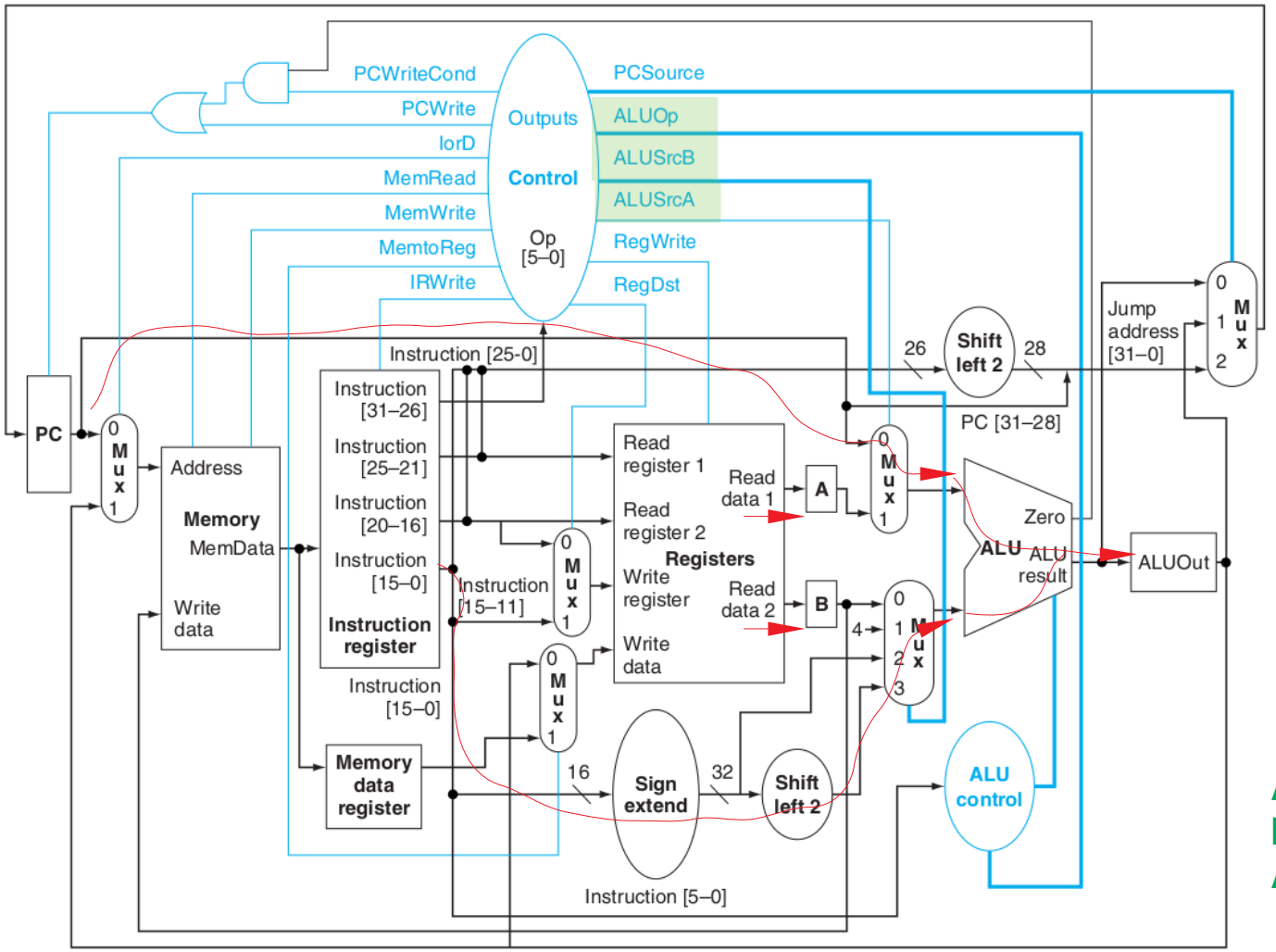
sw
1. Fetch

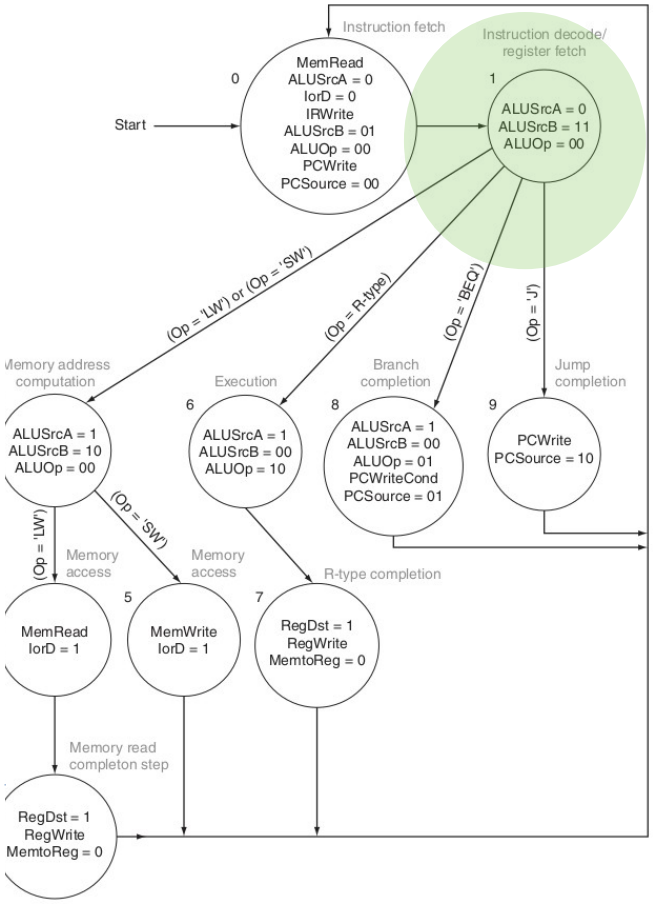




IR <= Memory[PC];
PC <= PC + 4;

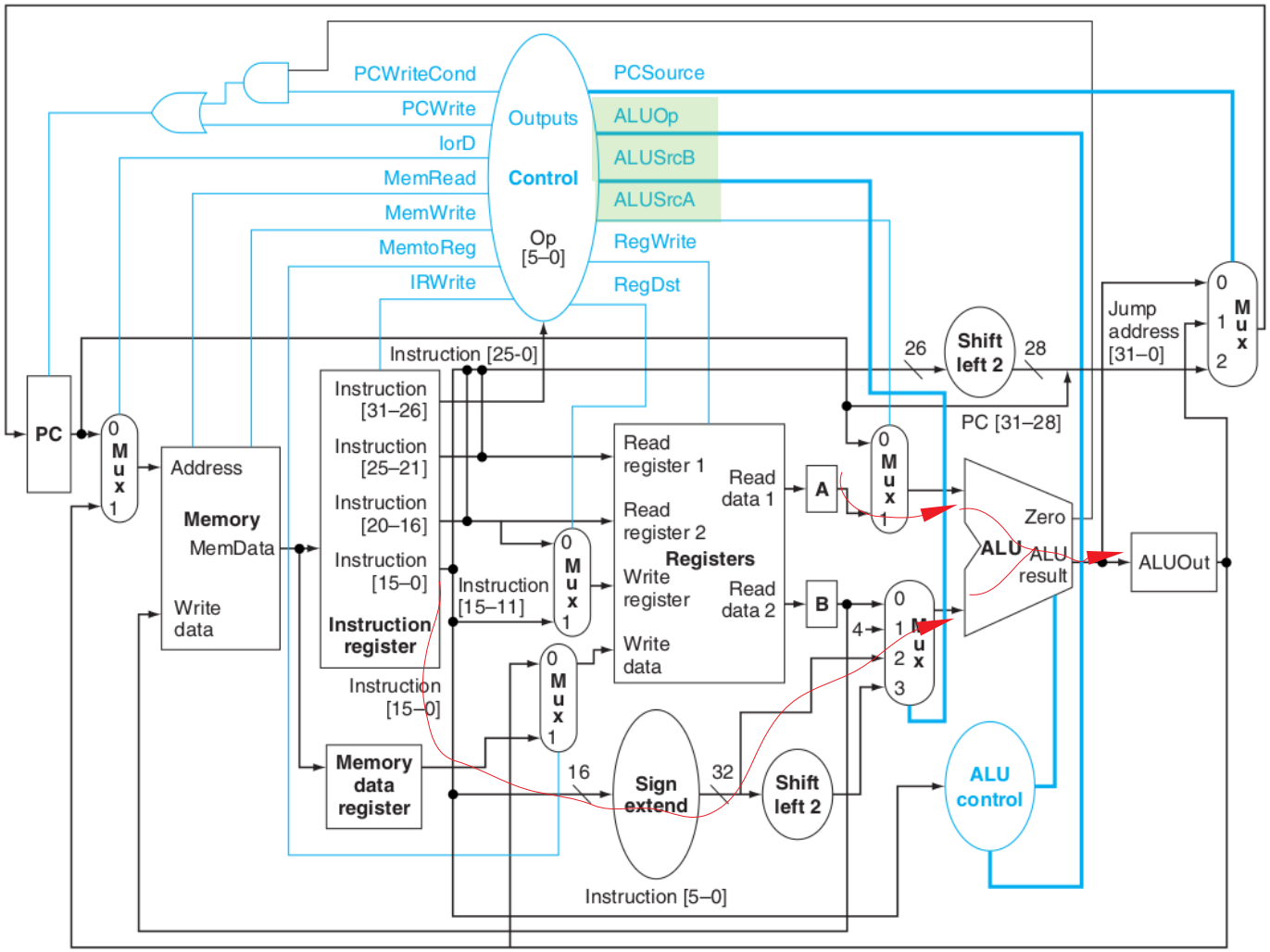
sw
2. Decode and operand fetch

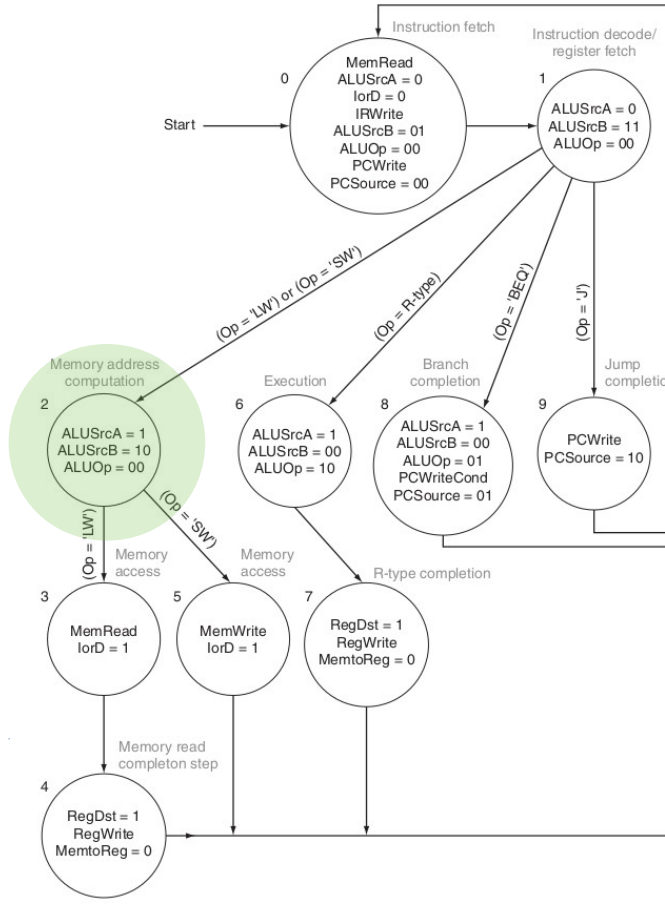





```
A <= Reg[IR[25:21]];
B <= Reg[IR[20:16]];
ALUOut <= PC +
      (sign-extend (IR[15-0]) << 2);
```

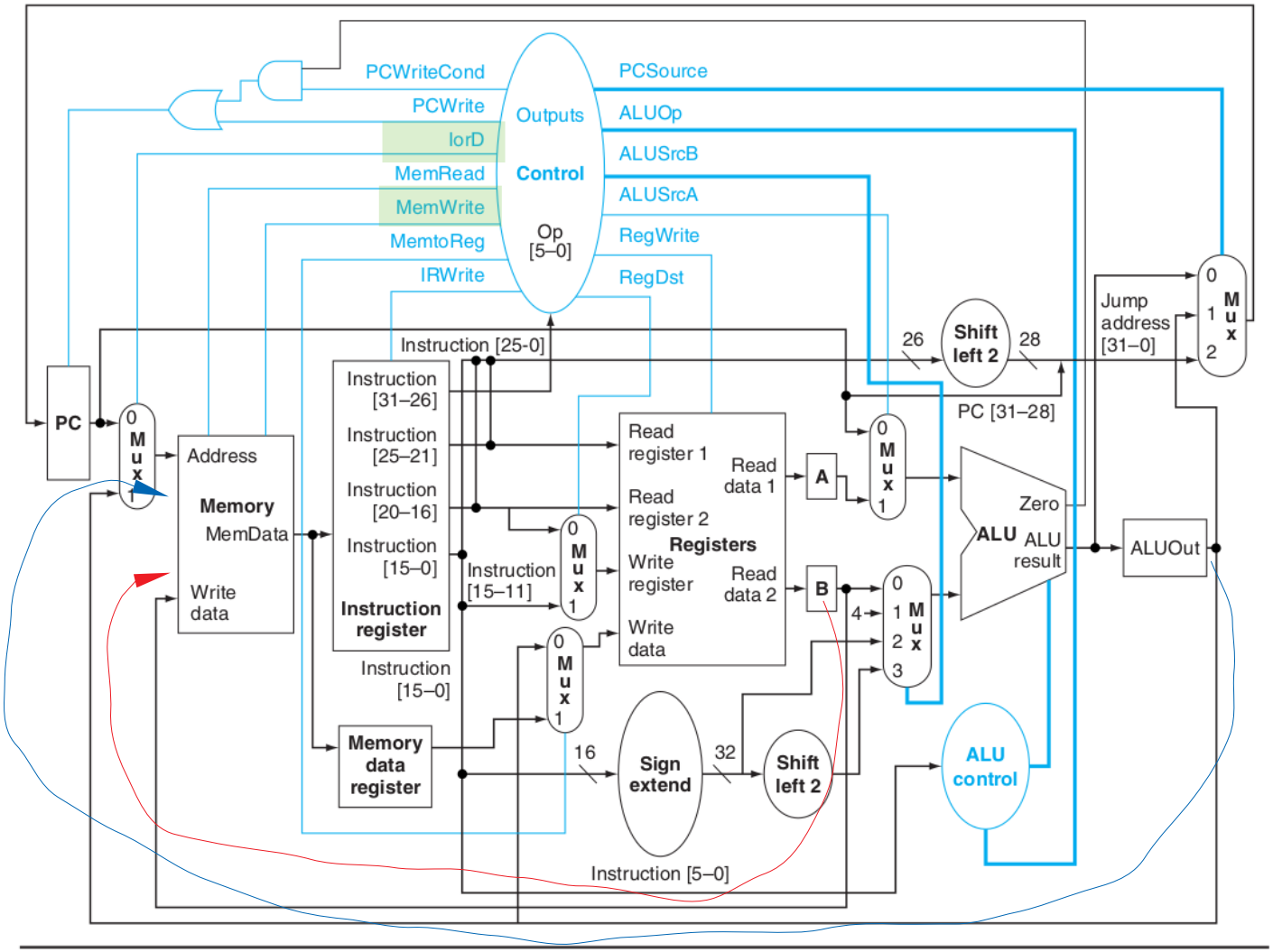
sw
3. Memory address computation

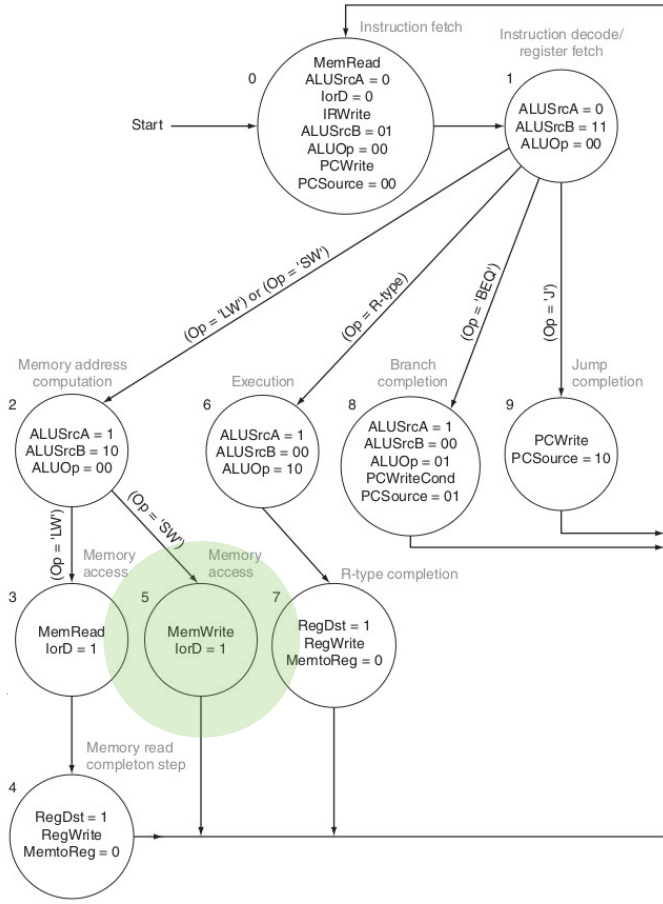




ALUOut <= A + sign-extend (IR[15:0]);

sw
4. Memory access

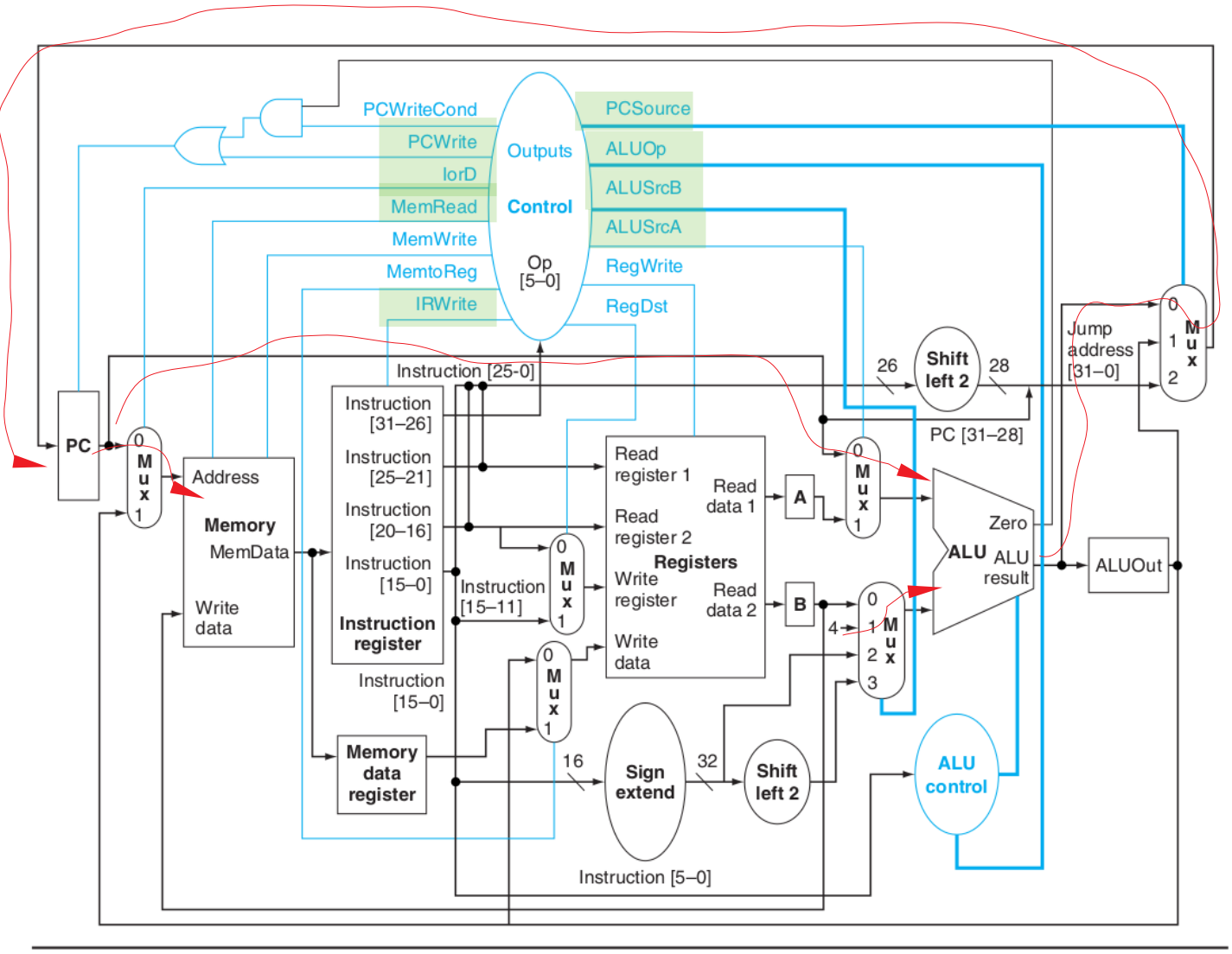


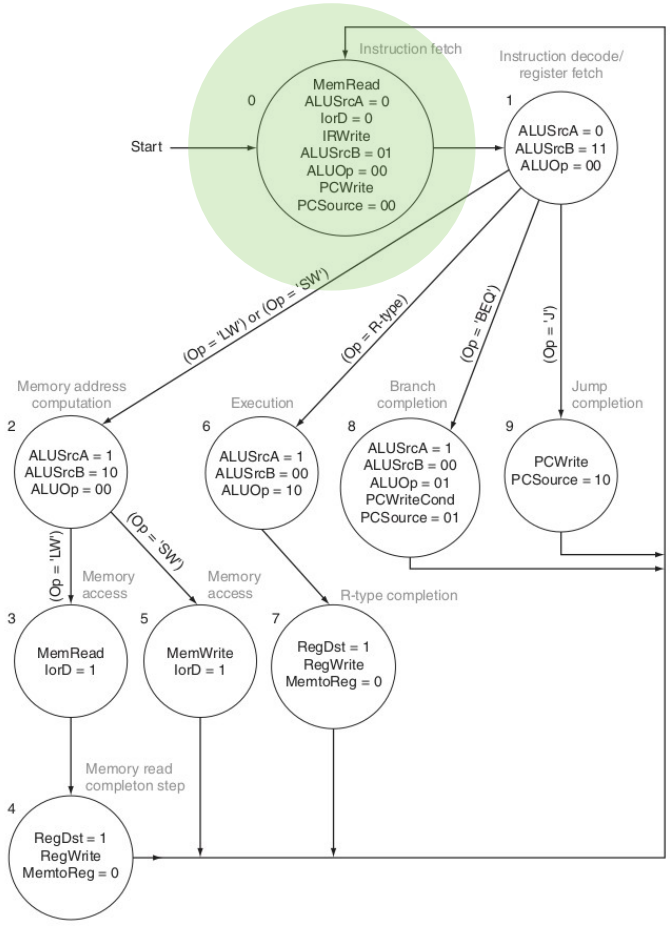


Memory [ALUOut] <= B;

lw $1, 0x10($2)

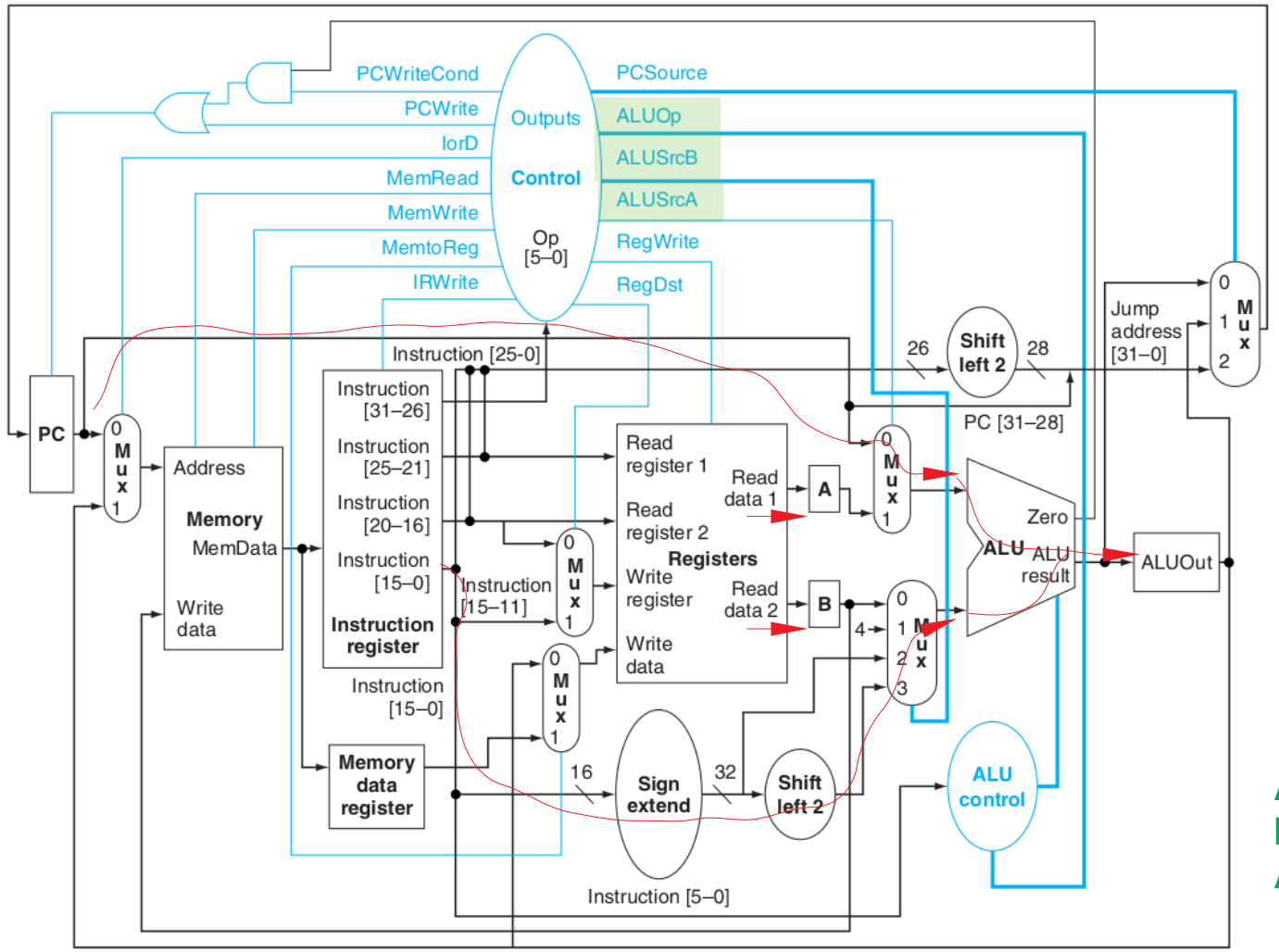
lw
1. Fetch

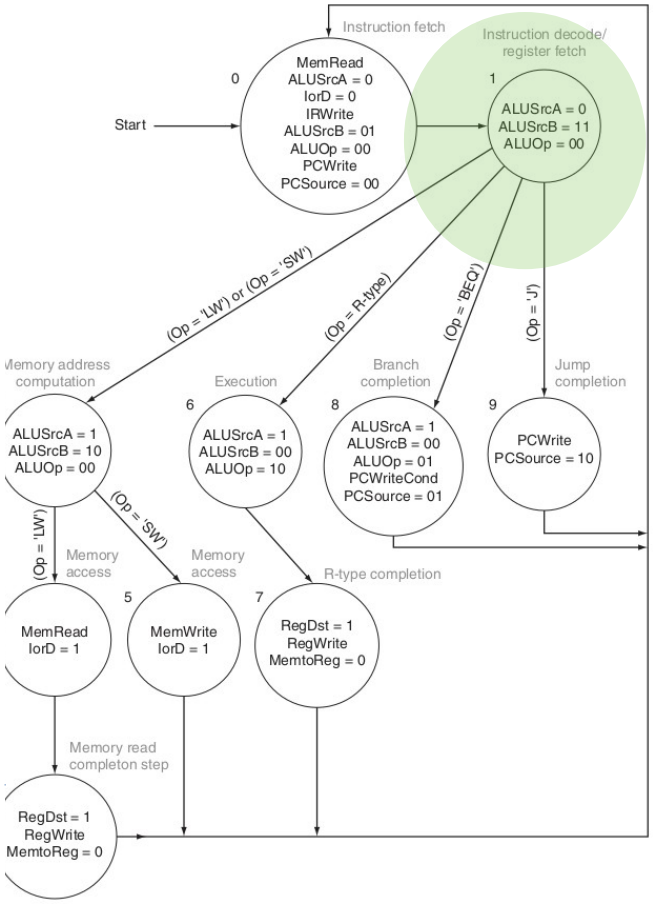




IR <= Memory[PC];
PC <= PC + 4;

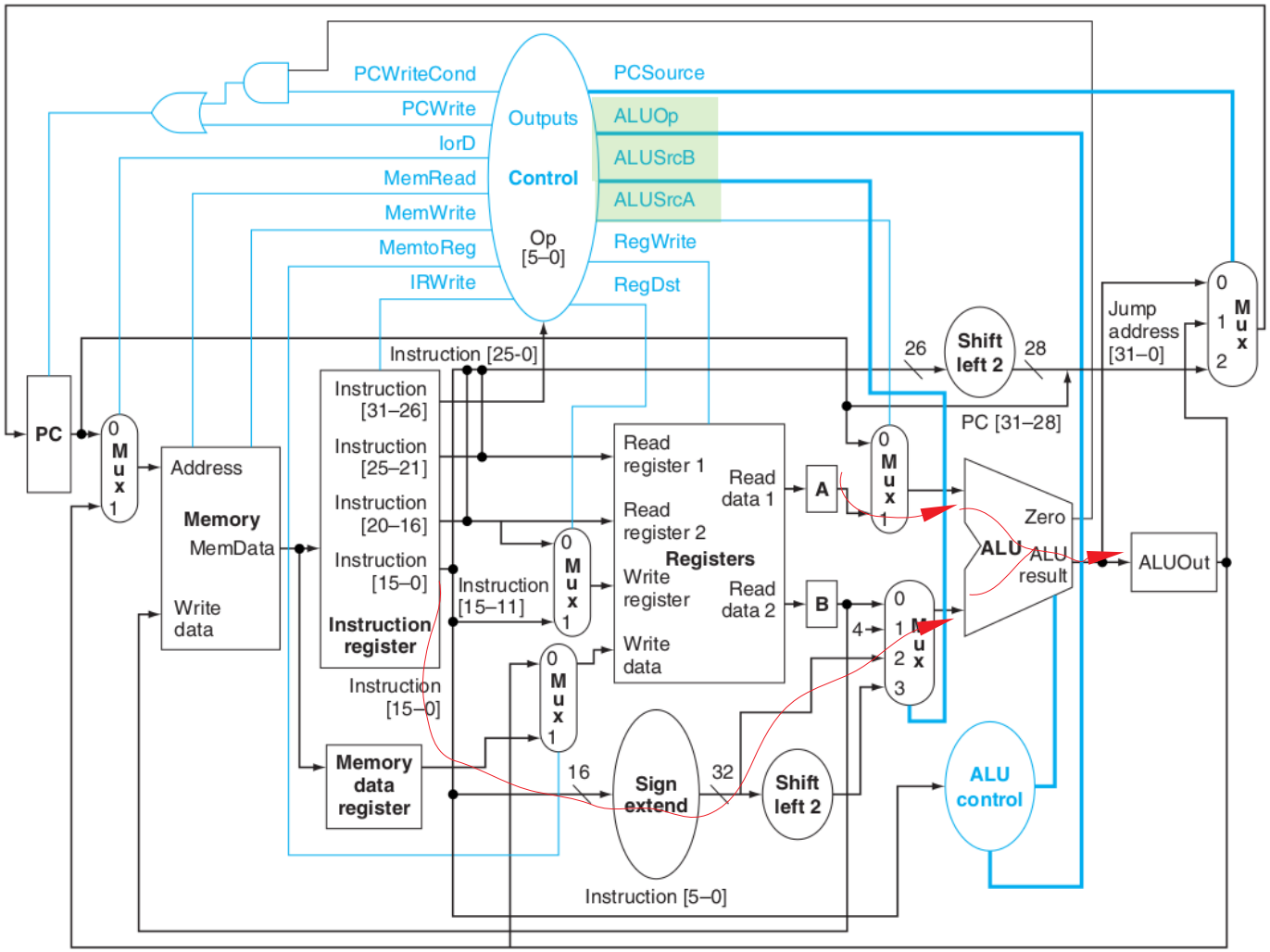
lw
2. Decode and operand fetch

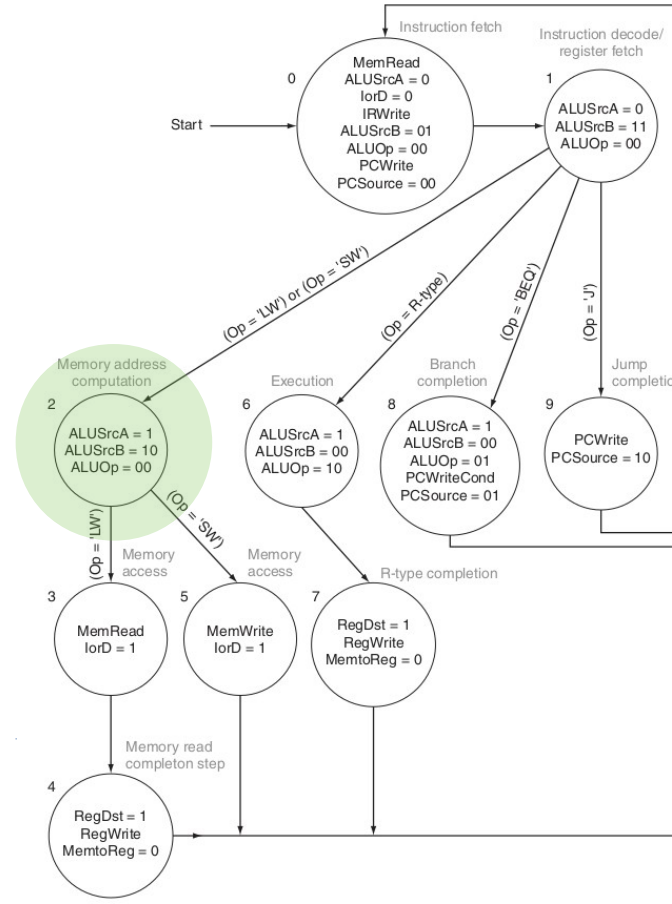





```
A <= Reg[IR[25:21]];
B <= Reg[IR[20:16]];
ALUOut <= PC +
      (sign-extend (IR[15-0]) << 2);
```

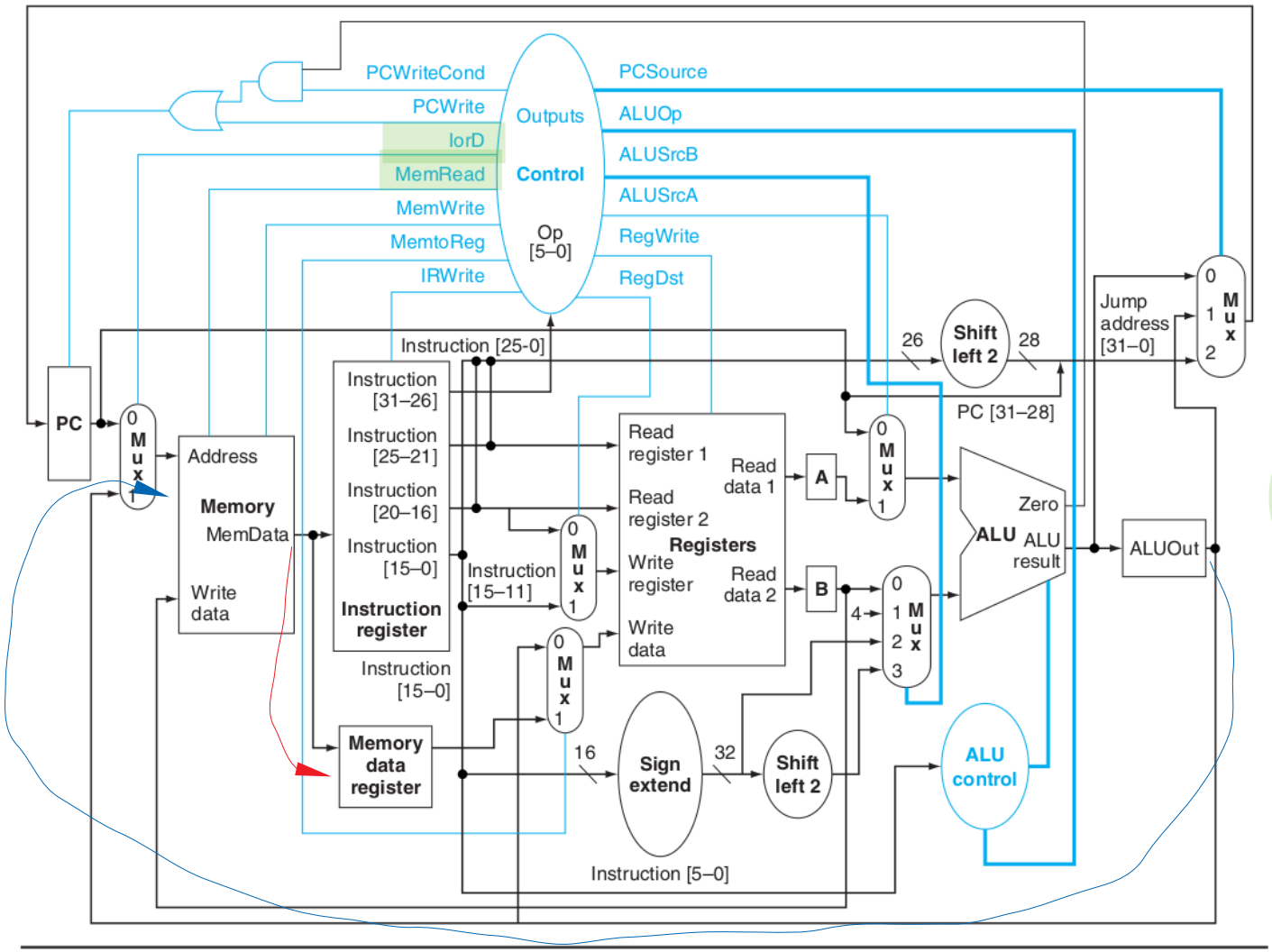
lw
3. Memory address computation

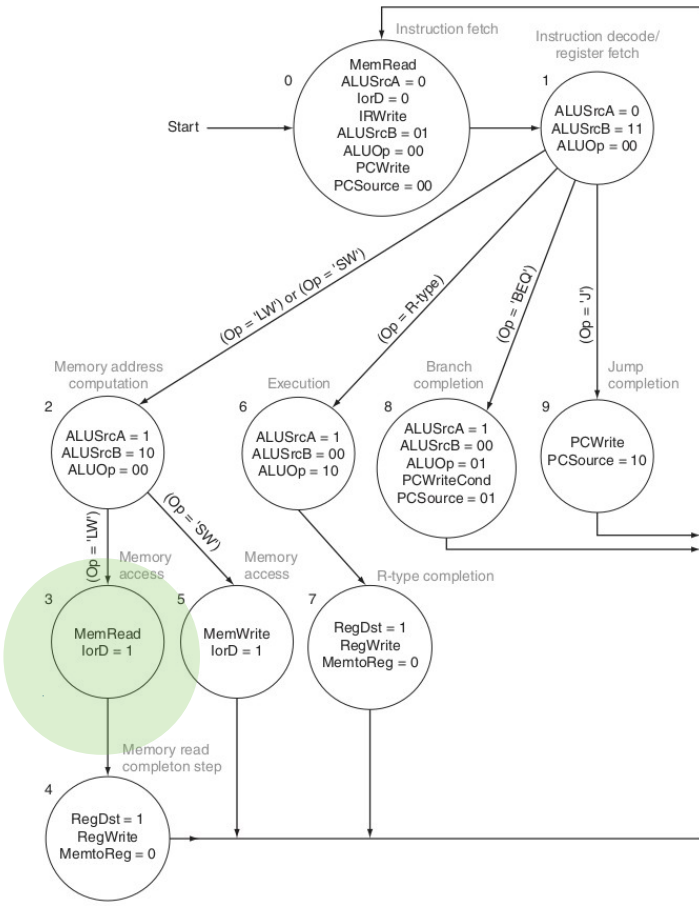




ALUOut <= A + sign-extend (IR[15:0]);

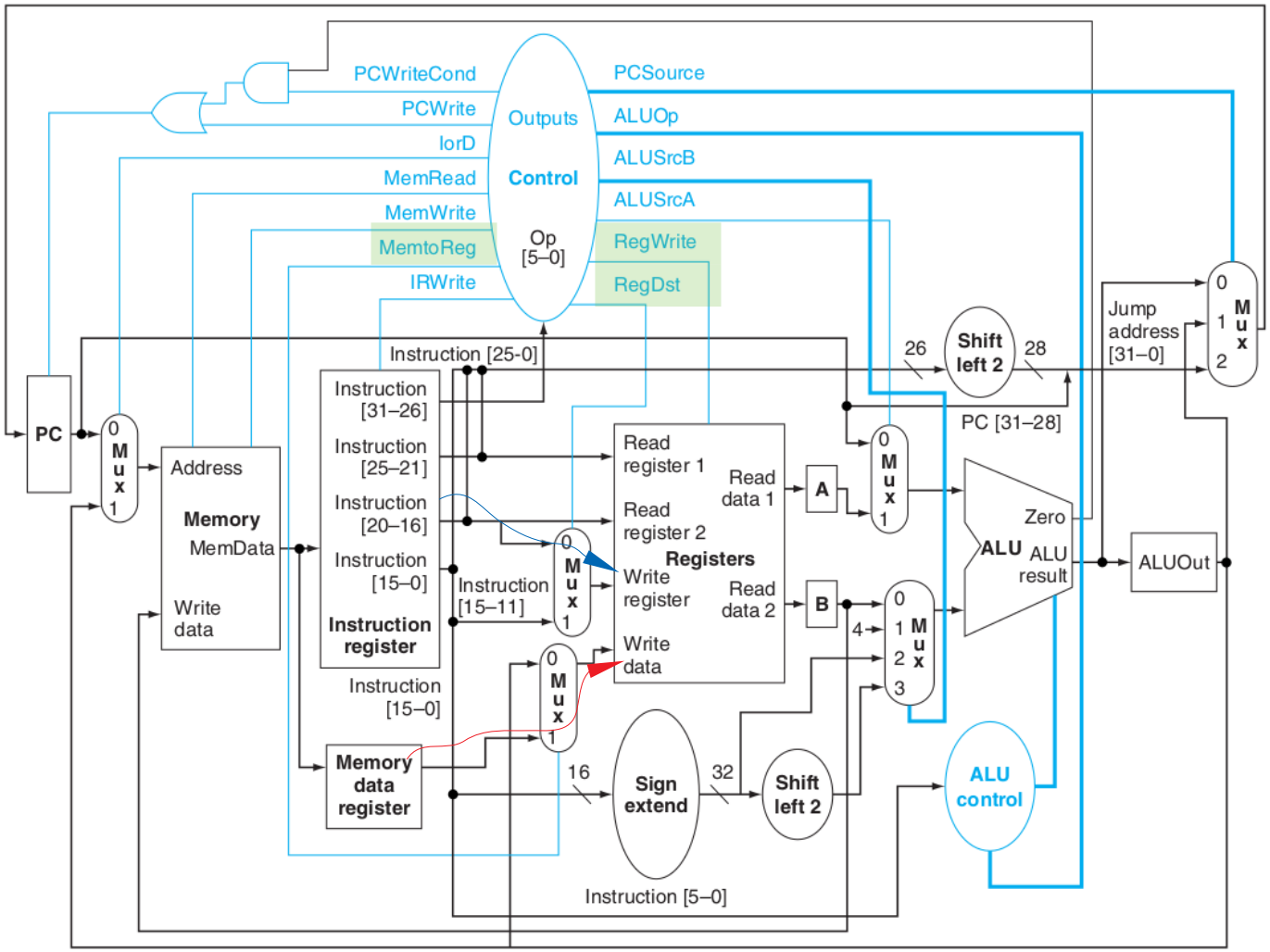
lw
4. Memory access

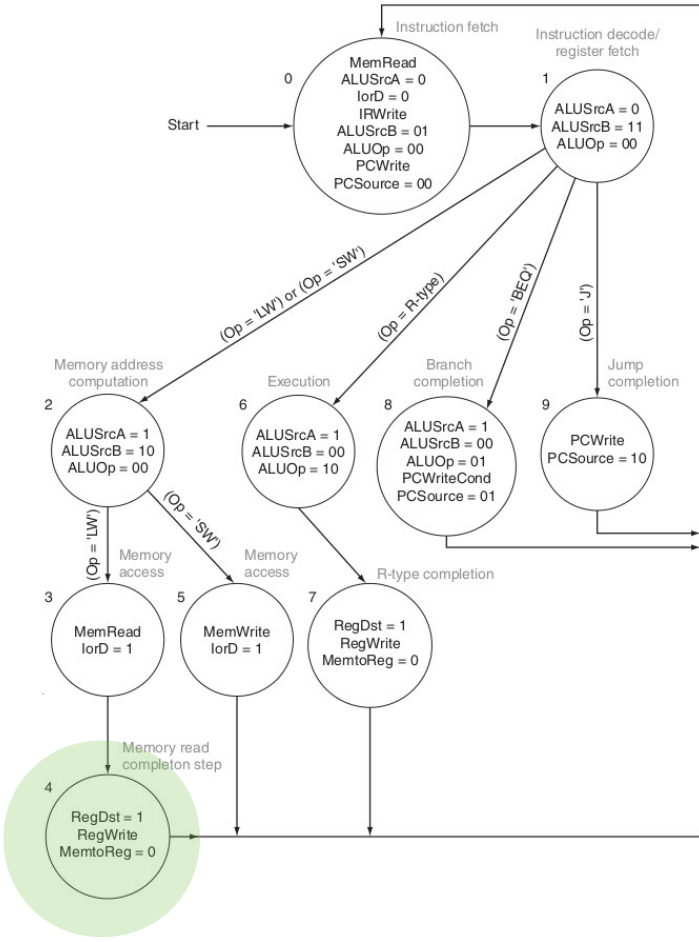




MDR <= Memory [ALUOut];

lw
5. Memory read completion





Reg[IR[20:16]] <= MDR;

Créditos

Figuras de Patterson&Hennessy 2018, 2004 – Morgan Kaufmann

CI1212 Arquitetura de Computadores

Prof. Eduardo Todt

todt@inf.ufpr.br

Período Especial 2020